



Bilkent University
Department of Computer Engineering

Senior Design Project

Team T2507

SCUBAMIND - MENTAL HEALTH

Design Project Final Report

22103231 Merve Güleç merve.gulec@ug.bilkent.edu.tr

22103775 Metin Çalışkan metin.caliskan@ug.bilkent.edu.tr

22101998 Muhammed Fatih Başal fatih.basal@ug.bilkent.edu.tr

22103416 Murathan Işık murathan.isik@ug.bilkent.edu.tr

22103406 Yiğit Koşum yigit.kosum@ug.bilkent.edu.tr

Hamdi Dibeklioglu
İlker Burak Kurt and Mert Bıçakçı

30.04.2026

Table of Contents

1. Introduction.....	5
2. Requirements Details.....	5
2.1 Functional Requirements.....	5
2.1.1 Data Collection and Sensing.....	5
2.1.2 Data Processing and Analysis.....	6
2.1.3 User Interface and Interaction.....	6
2.1.4 Privacy and Security.....	6
2.2 Non-Functional Requirements.....	7
2.2.1 Performance.....	7
2.2.2 Security.....	7
2.2.3 Usability.....	7
2.2.4 Reliability.....	7
2.2.5 Scalability and Maintainability.....	7
3. Final Architecture and Design Details.....	8
3.1 Current Software Architecture and Related Work.....	8
3.2 System Overview.....	9
3.3 Subsystem Decomposition.....	9
3.3.1 Flutter Module (UI Layer).....	10
3.3.2 Native Android Layer (Data & Storage).....	10
3.3.3 AI Module (Embedded Core).....	10
3.3.4 LLM Companion.....	11
3.4 Hardware / Software Mapping.....	11
3.5 Persistent Data Management.....	11
3.6 Access Control and Security.....	12
3.7 Method Level Design.....	12
3.7.1 Core Algorithm: Unified Daily Data Update.....	12
3.7.2 Core Algorithm: Weekly Risk Score Computation.....	13
3.7.3 Core Algorithm: On-Device ASR (Whisper).....	13
3.7.4 Class Diagram Overview.....	13
3.7.5 Sequence Diagram.....	14
3.7.6 Activity Diagram.....	15
3.7.7 Use Case Diagram.....	16
3.7.8 Data Flow Diagram.....	17
3.8 Development Methodology.....	18
4. Development / Implementation Details.....	19
4.1 Technology Stack.....	19
4.2 Subsystem Services.....	19
4.2.1 Mobile App Subsystem.....	19
4.2.2 Backend / Native Android Subsystem.....	20
4.2.3 ML / Analytics Subsystem.....	21
4.3 Mobile Application Implementation.....	22
4.4 Native Android Layer Implementation.....	23

4.5 AI Module Implementation.....	23
4.6 Flutter–Native Communication.....	24
4.7 Database Schema.....	24
4.8 LLM Companion Integration.....	24
4.9 Implemented Key Application Screens.....	25
5. Test Cases and Results.....	32
5.1 Functional Test Cases.....	32
TC-F01 Home Page Loads with Mental Health Overview.....	32
TC-F02 Pull-to-Refresh Reloads All Home Blocks.....	32
TC-F03 Voice Recordings List Loads from DB.....	32
TC-F04 Play/Pause Toggles Correctly.....	33
TC-F05 Confirm Delete Removes Recording.....	33
TC-F06 SpeechToTextModel Loads Encoder, Decoder, and Tokenizer.....	33
TC-F07 Inference Returns Empty String for Silent Audio.....	34
TC-F08 Chat Page Sends and Displays Messages.....	34
TC-F09 PHQ Questionnaire Loads and Submits.....	34
TC-F10 Onboarding Flow Completes and Is Not Shown Again.....	35
TC-F11 Transcription Section Shown for Completed Recordings.....	35
TC-F12 Transcribing Indicator Shown While Transcription Is Null.....	35
TC-F13 Inference Fails Gracefully When Model Not Loaded.....	36
TC-F14 Chat Page Loads and Displays Message List.....	36
TC-F15 Empty Message Is Not Sent.....	36
5.2 Non-Functional Test Cases.....	37
TC-P01 ASR Inference Completes Within 10 Seconds.....	37
TC-P02 Voice Recordings List Loads Within 2 Seconds for 50+ Entries.....	37
TC-P03 Home Page Initial Load Completes Within 3 Seconds.....	37
TC-S01 Voice Recordings Stored in Private App Directory.....	38
TC-S02 Unauthorized DB Access Attempt Is Blocked.....	38
TC-S03 PHQ Data Not Stored in Plain-Text Shared Preferences.....	38
TC-UX01 User Can Start a Recording Without Instructions.....	39
TC-UX02 Error Messages Are Clear and Actionable.....	39
TC-UX03 Interactive Elements Have Adequate Touch Targets.....	39
TC-R01 System Works Under Heavy Load (Concurrent Transcriptions).....	40
TC-R02 App Recovers from ASR Crash Without Data Loss.....	40
TC-R03 Data Persists After Unexpected App Termination.....	40
5.3 Test Results Summary.....	41
6. Maintenance Plan and Details.....	44
6.1 Corrective Maintenance.....	44
6.2 Adaptive Maintenance.....	45
6.3 Perfective Maintenance.....	45
6.4 Database Migration Protocol.....	46
6.5 Dependency Management.....	46
6.6 User's Manual and Installation Instructions.....	46
6.6.1 System Requirements.....	46

6.6.2 Installation.....	46
6.6.3 First Launch and Onboarding.....	46
6.6.4 Daily Usage.....	47
6.6.5 Data Deletion.....	47
7. Other Project Elements.....	47
7.1 Consideration of Various Factors in Engineering Design.....	47
7.1.1 Constraints.....	47
Public Health.....	47
Public Safety.....	47
Public Welfare.....	48
Global Factors.....	48
Cultural Factors.....	48
Social Factors.....	48
Environmental Factors.....	48
Economic Factors.....	48
7.1.2 Standards.....	49
IEEE Standards.....	49
ISO Standards.....	49
UML Standards.....	49
Healthcare Data Standards (FHIR / HL7).....	49
KVKK (Turkish Personal Data Protection Law).....	49
Object-Oriented Design Principles.....	50
Android Accessibility Guidelines.....	50
7.2 Ethics and Professional Responsibilities.....	50
7.2.1 Responsibility for User Well-Being.....	50
7.2.2 Privacy and Data Ethics.....	50
7.2.3 Transparency and Informed Consent.....	51
7.2.4 Avoiding Harm Through AI Model Outputs.....	51
7.2.5 Professional Responsibility in Documentation and Testing.....	51
7.2.6 Responsible Use of External AI Services.....	51
7.3 Teamwork Details.....	51
7.3.1 Contributing and Functioning Effectively on the Team.....	51
7.3.2 Helping Create a Collaborative and Inclusive Environment.....	52
7.3.3 Taking Lead Role and Sharing Leadership on the Team.....	53
7.3.4 Meeting Objectives.....	53
7.4 New Knowledge Acquired and Applied.....	54
7.4.1 On-Device Machine Learning and ONNX Runtime.....	54
7.4.2 Digital Phenotyping and Behavioral Signal Processing.....	55
7.4.3 Flutter Development and Flutter-Native Communication.....	55
7.4.4 Android Background Services and System-Level APIs.....	55
7.4.5 Privacy-Preserving System Design.....	55
7.4.6 Large Language Model Integration and Prompt Engineering.....	56
8. Conclusion and Future Work.....	56
8.1 Conclusion.....	56

8.2 Future Work.....	57
9. Glossary.....	57
10. References.....	59
11. Appendix — User Manual and Project Website.....	61

1. Introduction

Mental health disorders, particularly depression, represent a major global health challenge affecting millions of individuals worldwide. According to the World Health Organization, depression is one of the leading causes of disability globally and significantly affects individuals' daily functioning and quality of life.

Traditional mental health monitoring methods typically rely on periodic clinical assessments and self-reported questionnaires. While these approaches provide useful information, they often fail to capture the continuous and dynamic behavioral changes occurring in everyday life. As a result, important behavioral patterns related to mental well-being may remain undetected between clinical visits.

ScubaMind is designed as a privacy-preserving mobile mental health monitoring system that leverages digital phenotyping techniques. The system collects behavioral and physiological signals from smartphone sensors and wearable devices and analyzes these signals using embedded artificial intelligence models. A key design principle of the system is its local-first architecture, where sensitive behavioral data is processed directly on the user's device to ensure strong privacy protection.

It is important to emphasize that ScubaMind is not intended to function as a clinical diagnostic tool. Instead, it acts as a decision-support and awareness system that helps users monitor behavioral trends over time and encourages them to seek professional support when necessary.

This Final Report presents the complete system as implemented, covering refined requirements, final architecture, implementation details, test results, maintenance plan, and broader project considerations, including ethics, teamwork, and new knowledge acquired.

2. Requirements Details

The requirements of the ScubaMind system were initially identified and documented during the analysis phase of the project. Through iterative refinement during the design and implementation phases, these requirements were updated to reflect the final capabilities and constraints of the system.

2.1 Functional Requirements

2.1.1 Data Collection and Sensing

- FR-01: The system shall collect step count data using the device's built-in step counter sensor.
- FR-02: The system shall monitor sleep patterns using screen-on/off events and wearable-derived data via the Health Connect API.
- FR-03: The system shall collect GPS-based mobility data to derive spatial behavioral metrics such as the Radius of Gyration and Routine Index.
- FR-04: The system shall collect phone usage statistics, including app usage duration and screen unlock counts, using the Android UsageStats API.
- FR-05: The system shall provide an optional specialized keyboard (IME) to capture text input across applications for linguistic analysis.

- FR-06: The system shall support voice recording for prosody-based analysis.
- FR-07: The system shall integrate with Samsung Galaxy Watch via the Health Connect Bridge to ingest Heart Rate Variability (HRV) and sleep stage data.

2.1.2 Data Processing and Analysis

- FR-08: The system shall process collected behavioral data on-device using embedded machine learning models (TensorFlow Lite).
- FR-09: The system shall aggregate multimodal signals into a unified daily behavioral record (UnifiedDataEntity).
- FR-10: The system shall compute a weekly behavioral risk score based on mobility and multimodal behavioral analysis.
- FR-11: The system shall perform speech-to-text transcription on voice recordings using an on-device Whisper ASR model.
- FR-12: The system shall perform prosody analysis on voice recordings to extract acoustic features.
- FR-13: The system shall analyze text data from keyboard input for sentiment and linguistic features.

2.1.3 User Interface and Interaction

- FR-14: The system shall provide an onboarding flow that collects baseline information and requests necessary permissions.
- FR-15: The system shall present behavioral insights on a Home screen dashboard showing risk score, mood indicators, sleep, and activity summaries.
- FR-16: The system shall provide a Goals screen allowing users to create, view, and complete personal wellness goals.
- FR-17: The system shall provide a Chat screen with an LLM-powered conversational assistant (Gemini API).
- FR-18: The system shall allow users to view and playback voice recordings with transcription display.
- FR-19: The system shall allow users to complete the PHQ-8 questionnaire within the application.
- FR-20: The system shall provide a Privacy & Data settings screen for managing sensor permissions.

2.1.4 Privacy and Security

- FR-21: The system shall store all behavioral data in an AES-256-encrypted local Room database (SQLCipher).
- FR-22: The system shall store encryption keys in the Android Keystore System (TEE/StrongBox).
- FR-23: The system shall provide a Data Purge feature that cryptographically destroys all locally stored data.
- FR-24: The system shall allow users to revoke any individual sensor permission at any time.

- FR-25: Raw voice recordings and keyboard text shall never leave the device hardware.

2.2 Non-Functional Requirements

2.2.1 Performance

- NFR-01: ASR inference for a 30-second audio clip shall complete within 10 seconds on a mid-range Android device.
- NFR-02: The Home page shall fully load and display all data within 3 seconds of navigation.
- NFR-03: Voice recordings list with 50+ entries shall render within 2 seconds.
- NFR-04: Background services shall not cause visible UI jank (frame drops > 16 ms).

2.2.2 Security

- NFR-05: Voice recordings shall be stored in the private app directory, inaccessible to unprivileged third-party applications.
- NFR-06: Unauthorized database access attempts by third-party apps shall be blocked.
- NFR-07: PHQ-8 responses shall not be stored in plain-text SharedPreferences.

2.2.3 Usability

- NFR-08: A first-time user shall be able to locate and start a voice recording within 60 seconds without instructions.
- NFR-09: Error messages shall use plain language with clear recovery actions.
- NFR-10: All interactive touch targets shall meet the Android accessibility minimum of 48 x 48 dp.

2.2.4 Reliability

- NFR-11: The system shall remain stable and responsive when 5 voice recordings are queued for transcription simultaneously, with memory usage not exceeding 500 MB.
- NFR-12: The application shall recover from an ASR crash without data loss; voice recording metadata shall remain intact in the database.
- NFR-13: Data shall persist after an unexpected app termination (force-stop).

2.2.5 Scalability and Maintainability

- NFR-14: The modular architecture shall allow integration of new sensing modalities or ML models without major architectural changes.
- NFR-15: The database schema shall support migration without data loss using Room migration protocols.

3. Final Architecture and Design Details

The final architecture of the ScubaMind system is a layered, modular, and local-first mobile software architecture designed to support continuous behavioral monitoring, on-device AI analysis, and privacy-preserving data handling. The architecture separates user interaction, data collection, analytical processing, and optional conversational support into distinct subsystems.

3.1 Current Software Architecture and Related Work

Before defining the final ScubaMind architecture, existing digital mental health and behavioral monitoring systems were examined to identify their architectural strengths and limitations. These systems provided useful references for passive sensing, mood tracking, digital phenotyping, and wearable-based health monitoring; however, they also revealed important gaps related to privacy, multimodal integration, and on-device intelligence.

StudentLife is a research-oriented smartphone sensing system developed at Dartmouth College. It collects passive smartphone sensor data such as mobility, activity, phone usage, and sleep-related patterns to study the relationship between student behavior, academic performance, and mental health. Although StudentLife demonstrates the value of passive behavioral sensing, it is mainly a research data collection platform rather than a user-facing privacy-preserving mobile assistant. ScubaMind extends this idea by integrating passive sensing with wearable data, on-device AI inference, and user-facing behavioral feedback.

Moodpath is a mobile mental health application based mainly on self-reported mood tracking and questionnaire-based assessment. It provides structured mental health check-ins and feedback based on user responses. However, it depends heavily on manual input and may miss continuous behavioral changes that occur between questionnaire sessions. ScubaMind differs from this approach by combining self-reported information with passive behavioral signals collected from the smartphone and wearable devices.

Mindstrong Health represents a digital mental health platform that analyzes smartphone interaction patterns, such as typing behavior and device usage, to infer cognitive and behavioral markers. Its focus on interaction-based signals is relevant to ScubaMind's keyboard and phone usage analysis components. However, ScubaMind broadens the sensing scope by combining interaction data with mobility, sleep, activity, physiological signals, voice recordings, and multimodal AI-based risk analysis.

Google Fit provides general health and fitness data such as activity, sleep, and heart-related measurements. These platforms are useful for collecting physiological and activity-related information, but they are not specifically designed to produce mental health-oriented behavioral risk insights. ScubaMind uses similar health and activity data sources as inputs, but interprets them through a mental well-being monitoring perspective.

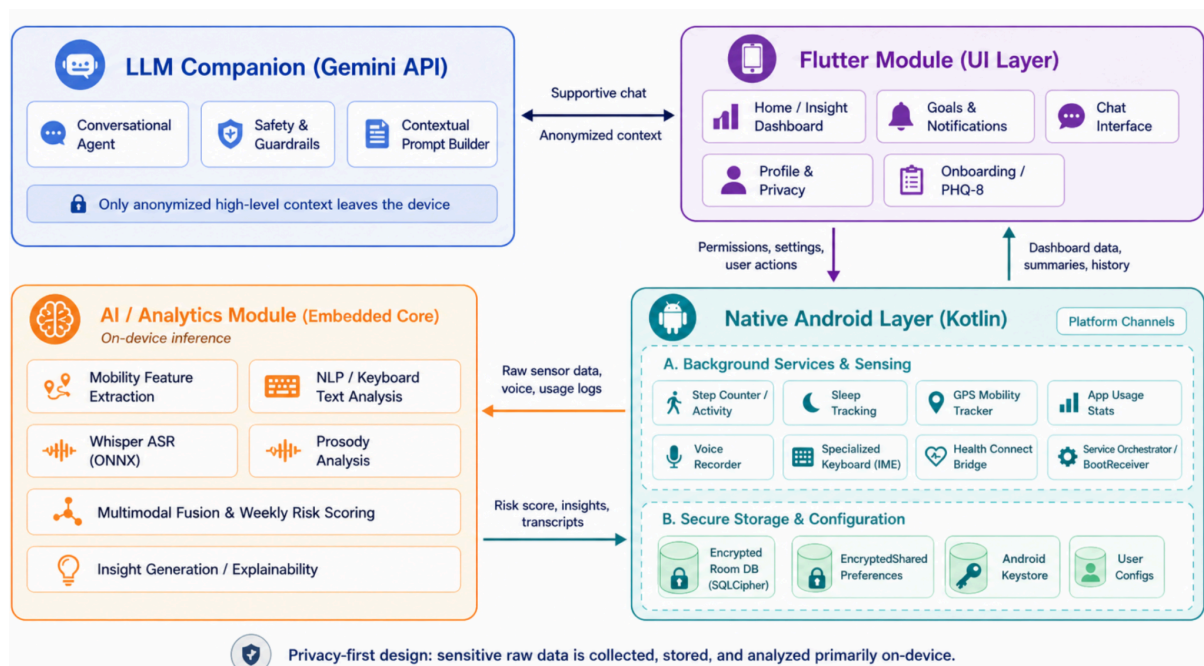
Overall, existing systems either focus on self-reporting, single-modal behavioral sensing, general fitness tracking, or cloud-based digital mental health analysis. ScubaMind addresses these limitations by combining multimodal behavioral sensing,

local encrypted storage, on-device AI analysis, and a privacy-preserving local-first architecture. This related work analysis shaped the final architecture by motivating the separation between the Flutter UI layer, Native Android sensing layer, AI module, and optional LLM Companion.

3.2 System Overview

The ScubaMind system is built using four primary, loosely coupled subsystems: (1) the Flutter Module (UI Layer), (2) the Native Android Layer (Data & Storage), (3) the AI Module (Embedded Core), and (4) the LLM Companion. All sensitive behavioral and physiological data are collected, stored, and analyzed primarily on the user's device in accordance with the local-first design principle.

At the top of the architecture, the Flutter Module provides the main user-facing interface, including the Insight Dashboard, AI Chat Interface, onboarding screens, goal management, and permission control components. The Native Android Layer acts as the operational backbone, managing background services for data collection and secure local persistence. The AI Module transforms raw behavioral and physiological signals into actionable mental health insights using embedded TensorFlow Lite models. The LLM Companion provides an optional conversational interface powered by the Gemini API.



[Figure 1 – High-level architecture diagram of the ScubaMind system.]

3.3 Subsystem Decomposition

The ScubaMind system is architected into four primary, loosely coupled subsystems. This modularity ensures that heavy computational tasks (AI inference) do not block the user interface and that background data collection remains resilient to OS-level process management.

3.3.1 Flutter Module (UI Layer)

Purpose: To serve as the primary gateway for user interaction, providing a transparent and intuitive interface for mental health monitoring and data sovereignty.

The Flutter Module is responsible for Data Visualization (transforming complex longitudinal datasets into human-readable charts on the Insight Dashboard), Privacy Management (providing a centralized hub for the user to grant, revoke, or audit sensor permissions), User Onboarding (guiding the user through the initial setup including personal baseline establishment and wearable synchronization), and Session Management (handling the state of the conversational interface during interactions with the LLM Companion).

The module receives processed Risk Scores, Mood Predictions, and Behavioral Insights from the AI Module via the Native Layer, and sends user-configured settings, manual mood entries, and chat transcripts to the Native Layer for local storage and further analysis.

3.3.2 Native Android Layer (Data & Storage)

Purpose: To act as the low-level operational backbone of ScubaMind, managing continuous, energy-efficient data acquisition and ensuring secure local persistence.

The Native Android Layer manages the lifecycle of Foreground Services through persistent notifications to prevent OS-level termination. It hosts the Background Service Orchestrator that monitors SharedPreferences for configuration changes initiated by the Flutter UI. When a user toggles a sensor in the settings, the orchestrator dynamically starts or stops the corresponding background service.

Background services include: the Mobility Manager (polling GPS and Accelerometer data to calculate spatial variance and Radius of Gyration), the UsageStats Manager (monitoring digital behavior patterns including Screen On/Off events and App Usage Statistics), the Specialized IME Keyboard (capturing raw text input across apps for linguistic analysis), and the Health Connect Bridge (synchronizing HRV and Sleep Stage physiological metrics from Samsung wearables). Data is persisted using an AES-256-encrypted SQLite database via the Room Persistence Library and SQLCipher.

3.3.3 AI Module (Embedded Core)

Purpose: To transform raw behavioral, linguistic, and physiological signals into actionable mental health insights through privacy-preserving, on-device machine learning.

The AI Module performs Digital Phenotyping by processing raw mobility traces into behavioral biomarkers such as the Routine Index. Its Linguistic and NLP Unit performs both textual analysis (sentiment, emotional valence, linguistic complexity) and prosody analysis (extracting acoustic features from voice recordings such as pitch variance, speech rate, and intonation). A late-fusion neural network synthesizes phone behavior, text analysis, prosody features, and wearable physiological data into a unified Depression Risk Score. All inference is conducted locally using TensorFlow Lite, ensuring that audio processing and linguistic analysis are completed without external data transmission.

3.3.4 LLM Companion

Purpose: To act as a supportive Psychological Assistant by integrating the Gemini LLM API to facilitate supportive dialogue and encourage reflective Audio Journaling.

The LLM Companion dynamically constructs prompts for Gemini by aggregating the latest behavioral insights (e.g., reduced activity levels, disrupted sleep patterns). It specifically instructs the model to guide users through daily diary sessions, facilitating the collection of high-quality audio for prosody analysis. A Safety and Guardrail Layer uses a combination of Gemini's built-in safety filters and custom keyword monitoring to detect high-risk intent and trigger appropriate emergency protocols. Only anonymized, high-level behavioral context is sent to the external API; raw data never leaves the device.

3.4 Hardware / Software Mapping

ScubaMind utilizes a hybrid mapping strategy. All sensitive data acquisition, pre-processing, and behavioral analysis are mapped to the user's local smartphone and Samsung wearable hardware. Only the high-level conversational logic is mapped to the cloud via the Gemini API.

Subsystem	Hardware Component	Software Stack / Technology
Flutter Module (UI)	Smartphone Display & CPU	Flutter SDK (Dart)
Passive Sensing	Smartphone GPS, Accelerometer, Step Counter	Android SDK (Kotlin), UsageStats API
Wearable Integration	Samsung Galaxy Watch (Wear OS)	Samsung Health / Health Connect API
AI Module (Inference)	Smartphone NPU / GPU	TensorFlow Lite (TFLite)
Speech Recognition	Smartphone CPU/NPU	Whisper ONNX (on-device ASR)
Data Persistence	Smartphone Flash Storage	Room DB, SQLCipher (AES-256)
Conversational Agent	Cloud Infrastructure	Gemini API (REST / WebSocket)

Table 1 – Component Mapping Table

3.5 Persistent Data Management

ScubaMind utilizes a Local-First persistence strategy to ensure user privacy and data sovereignty. All raw and processed data are stored within the smartphone's internal storage using a highly secure, encrypted relational database built on the Room Persistence Library with SQLCipher for AES-256 page-level encryption.

The database passphrase is dynamically generated as a UUID and stored in EncryptedSharedPreferences, which is anchored to the Android Keystore System.

This ensures the master key is protected by hardware-backed security (TEE/StrongBox), making it inaccessible even to a rooted operating system.

The primary database entities include: UnifiedDataEntity (consolidating daily metrics including step counts, sleep duration, PHQ-8 scores, and keyboard text fragments), LocationSampleEntity and MobilityDailyMetricsEntity (handling GPS coordinates and derived spatial biomarkers), VoiceRecordingEntity (managing metadata for local audio files), and OnboardingEntity (storing baseline demographic data and initial psychological assessments). The system is currently at database Version 8, with a rigorous Migration Protocol ensuring stability across schema evolutions.

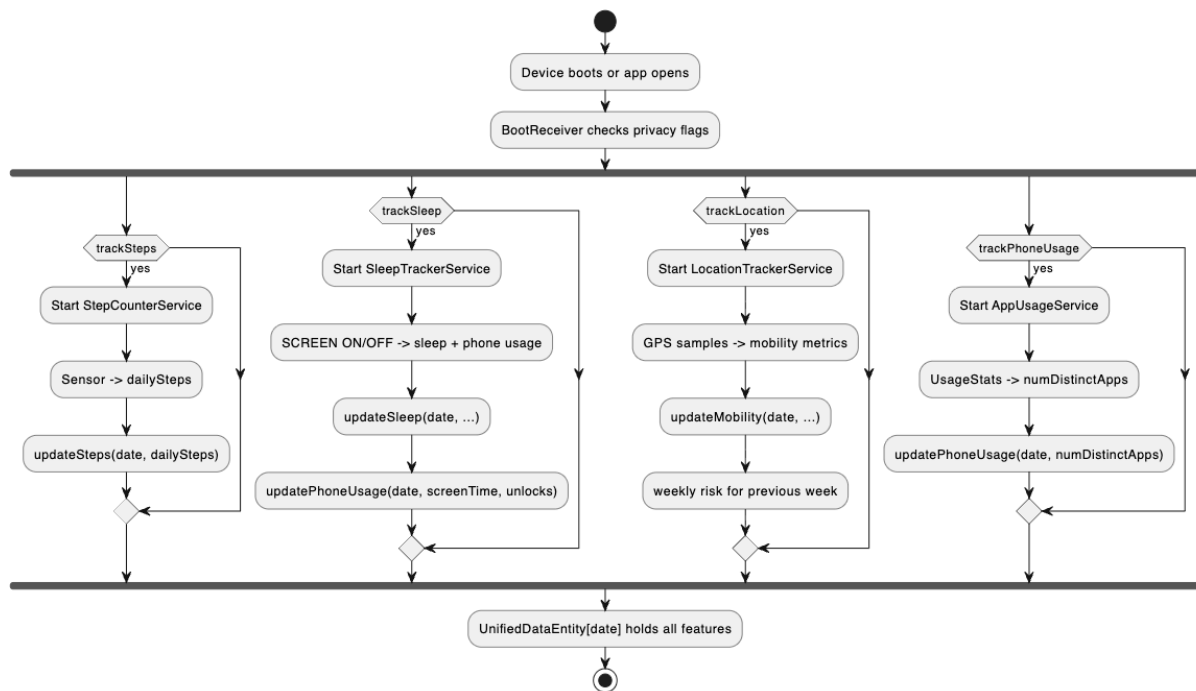
3.6 Access Control and Security

ScubaMind implements a Defense-in-Depth security architecture with multiple protection layers. At-rest encryption is achieved through AES-256 via SQLCipher with keys stored in hardware-backed Android Keystore. Android Application Sandbox isolation prevents other applications from accessing ScubaMind's internal storage, keyboard data, or voice recordings. Users must explicitly authorize individual data streams and can revoke permissions at any time through the Privacy & Permission Hub, which immediately stops the corresponding background service. The Data Purge feature provides cryptographic deletion by destroying both the database and its encryption keys, rendering all data permanently irrecoverable.

3.7 Method Level Design

3.7.1 Core Algorithm: Unified Daily Data Update

When the device boots or the application starts, the BootReceiver checks the user's privacy settings to determine which sensing modules are allowed to run. Based on these permissions, different background services (StepCounterService, SleepTrackerService, LocationTrackerService, AppUsageService) are activated. Each service collects a specific type of behavioral data and updates the corresponding fields in the UnifiedDataEntity through repository update functions. All collected modalities are aggregated into a single daily record for further analysis and model processing.



[Figure 2 – Unified Daily Data Update Diagram – see system documentation]

3.7.2 Core Algorithm: Weekly Risk Score Computation

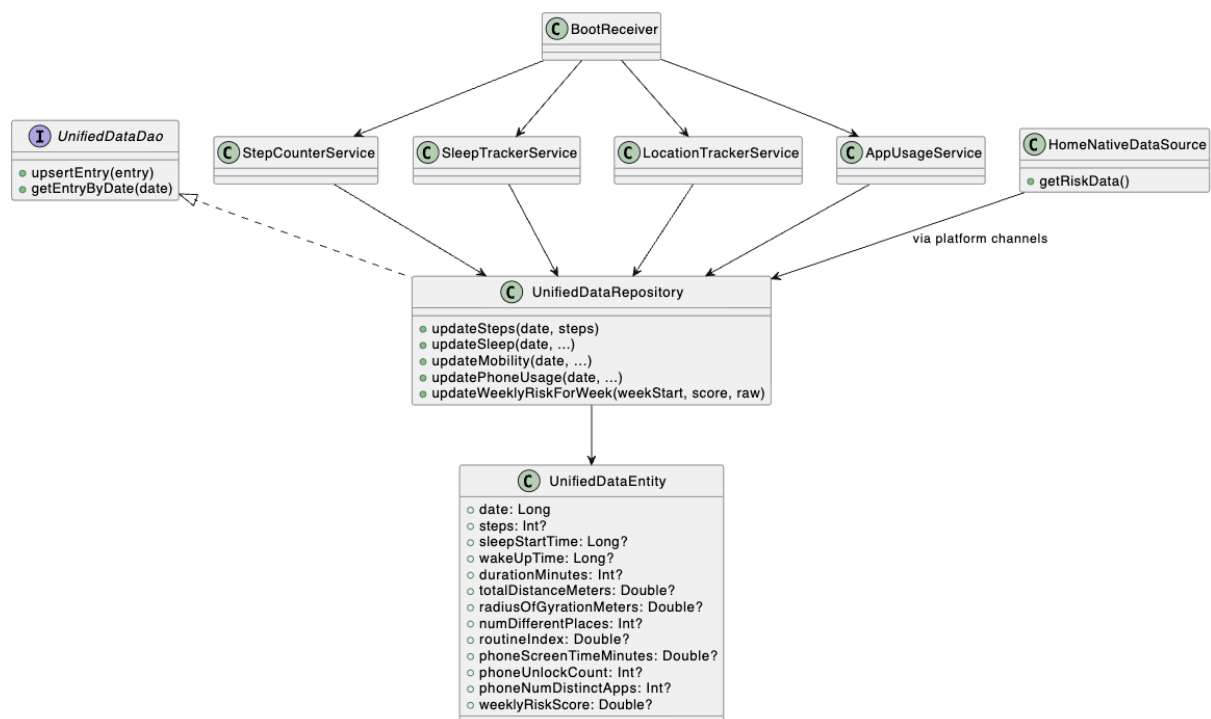
The weekly risk score is computed by the ML/Analytics subsystem using the mobility data aggregated in the UnifiedDataEntity. The system derives behavioral biomarkers such as Radius of Gyration, Routine Index, and total distance metrics over a 7-day rolling window. These features are fed into the late-fusion neural network alongside digital usage patterns and wearable physiological data to produce the final risk score. The score is then mapped to a qualitative risk level (Low, Moderate, High), and an explanatory message is presented to the user on the Home screen.

3.7.3 Core Algorithm: On-Device ASR (Whisper)

Voice recordings captured through the application are processed by the SpeechToTextModel, which uses ONNX Runtime to run the Whisper-tiny model directly on the device. The inference pipeline first checks the audio RMS level against a silence threshold (0.001f) and returns an empty transcription for silent audio segments. For audio with speech content, the encoder and decoder models are applied sequentially, with the WhisperTokenizer converting token outputs to text. The entire pipeline runs asynchronously to avoid blocking the UI thread.

3.7.4 Class Diagram Overview

The core Android data model centers on UnifiedDataEntity, which stores one row of daily unified features (steps, sleep, mobility, phone usage, and weekly risk score). UnifiedDataDao is the Room DAO, and UnifiedDataRepository is the main abstraction used by the four background services: StepCounterService, SleepTrackerService, LocationTrackerService, and AppUsageService. BootReceiver starts services after device boot based on stored user privacy settings. On the Flutter side, HomeNativeDataSource reads aggregated risk information through platform channels that query the same repository.



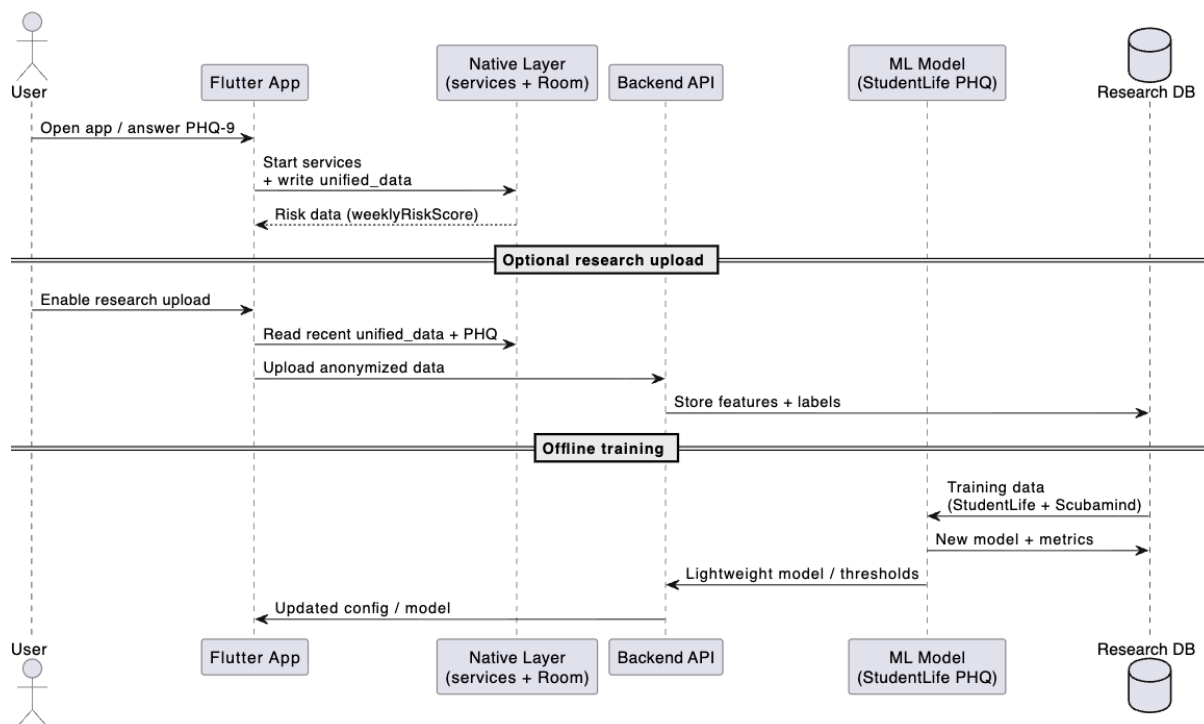
[Figure 3 – Class Diagram – Core data & services. See system documentation for full diagram.]

3.7.5 Sequence Diagram

The sequence diagram illustrates the interaction between the main components of the ScubaMind system during data collection, analysis, and feedback generation. When the user opens the application or completes the PHQ questionnaire, the Flutter application communicates with the native Android layer through platform channels. The native layer starts the required background services according to user permissions and stores the collected behavioral data in the local encrypted database.

After sufficient behavioral data is collected, the AI module processes the stored data and computes risk-related indicators such as weekly behavioral risk scores. These results are written back to the local database and made available to the Flutter interface. When the user opens the Home screen, the application retrieves the latest risk information and displays it as understandable feedback.

The sequence also shows the optional research participation flow. If the user explicitly enables research participation, anonymized behavioral data and PHQ labels may be uploaded to a research backend for offline model training. Updated lightweight model parameters or calibration thresholds can later be sent back to the application configuration. This optional flow is separated from the core local-first pipeline so that the main application remains functional without cloud connectivity.



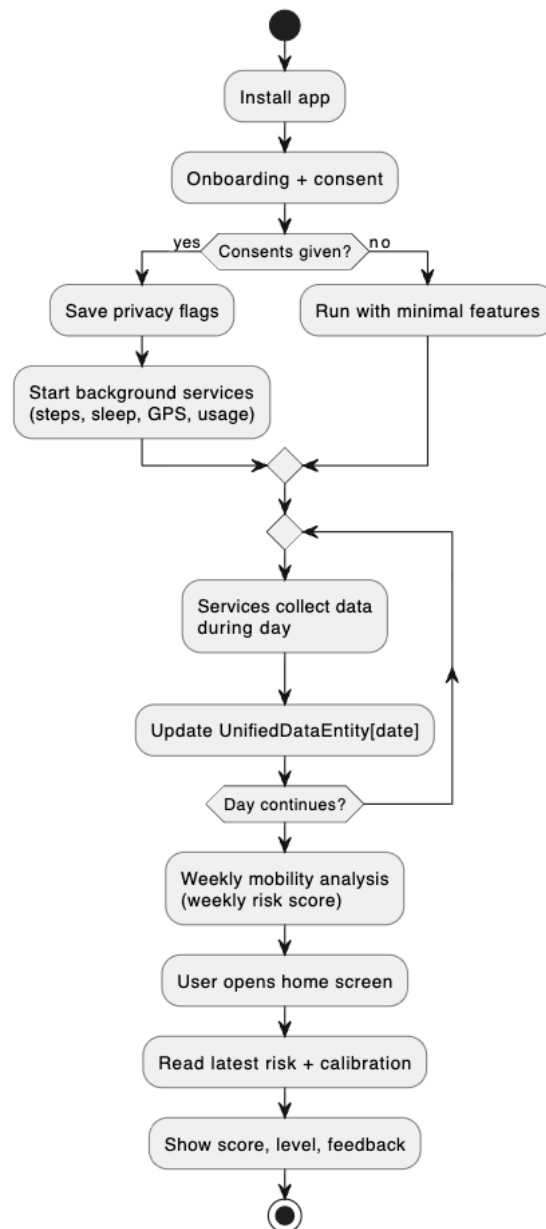
[Figure 4 – Sequence Diagram – Sequence Diagram of the ScubaMind System.]

3.7.6 Activity Diagram

The activity diagram represents the end-to-end user and system workflow of ScubaMind. After installing and launching the application, the user completes the onboarding process and reviews the consent screens. If the user grants the required permissions, the application stores the selected privacy preferences and activates the corresponding background services, such as step tracking, sleep monitoring, GPS-based mobility tracking, and phone usage monitoring.

During daily use, these background services collect behavioral signals and update the UnifiedDataEntity for the corresponding date. If the user denies some permissions, the system continues with limited functionality using only the available data sources. This supports the graceful degradation principle of the system.

Periodically, the collected daily records are aggregated and used to compute weekly behavioral risk indicators. When the user opens the Home screen, the application reads the latest risk score, risk level, and explanation from the local data layer and presents them through the dashboard. This flow demonstrates how ScubaMind connects consent management, passive sensing, local processing, and user-facing feedback in a privacy-preserving way.



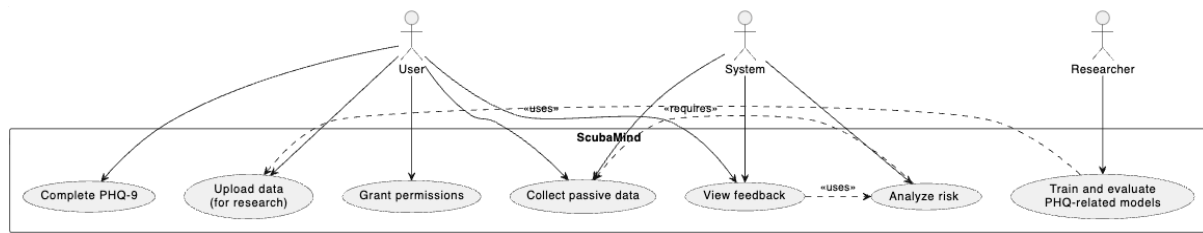
[Figure 5 – Activity Diagram – Activity Diagram: User Flow of the ScubaMind System]

3.7.7 Use Case Diagram

The use case diagram summarizes the main actors and high-level functions of the ScubaMind system. The primary actor is the user, who can complete onboarding, grant or revoke permissions, allow passive data collection, complete the PHQ questionnaire, view behavioral feedback, manage goals, use voice journaling, and interact with the LLM Companion.

The system actor represents the mobile application, native background services, local database, and AI module. It performs passive data collection, stores data securely, analyzes behavioral signals, computes risk-related indicators, and presents feedback to the user. The researcher actor is included only for the optional research participation scenario, where anonymized data can be used for model training and evaluation after explicit user consent.

The use case relationships show that risk analysis depends on passive data collection and that feedback generation depends on the output of the analysis module. This diagram helps clarify the boundary between user-controlled actions, automatic system behavior, and optional research-related functionality.



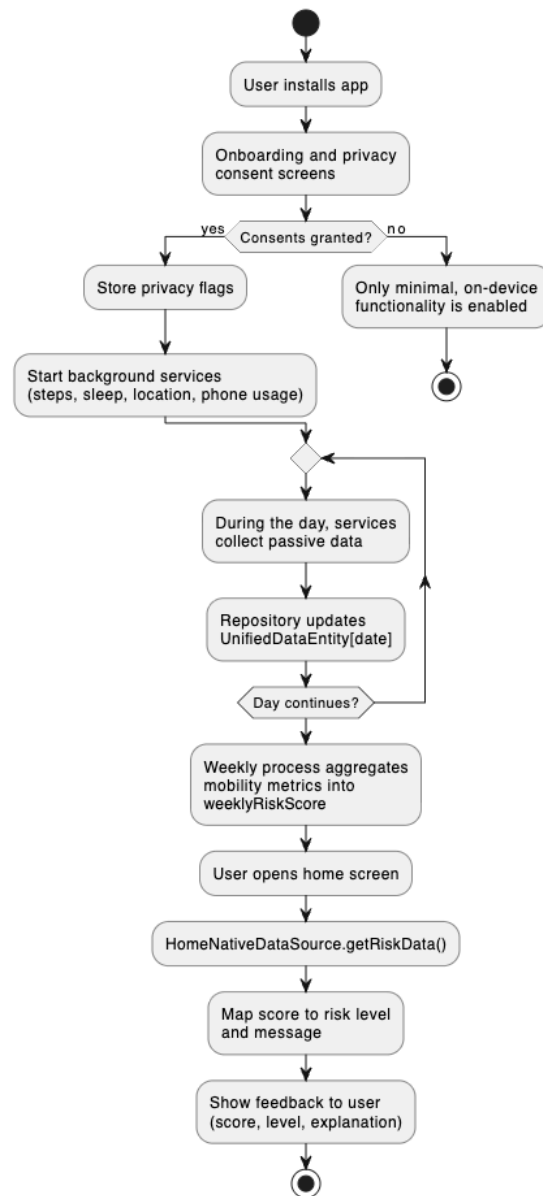
[Figure 6 – Use Case Diagram – Use Case Diagram: Main Actors and Use Cases]

3.7.8 Data Flow Diagram

The data flow diagram explains how information moves through the ScubaMind system from user consent to final feedback. The flow begins with onboarding and privacy consent. The user's permission choices are stored as privacy flags, and these flags determine which background services are allowed to run.

If consent is granted, Android services collect passive behavioral data such as steps, sleep patterns, GPS-based mobility traces, phone usage statistics, and optional voice or keyboard-based inputs. These raw and intermediate data are written to local repositories and stored in the encrypted Room database. The database then provides structured daily records to the AI and analytics layer.

The analytics layer transforms raw records into behavioral features, computes weekly risk indicators, and maps the numerical score into a qualitative risk level. The resulting score, level, and explanation are stored locally and retrieved by the Flutter UI when the user opens the Home dashboard. The diagram demonstrates the local-first flow of the system: sensitive raw data remains on the device, while only high-level summaries are used for user feedback or optional LLM context.



[Figure 7 – Data Flow Diagram – Data Flow Diagram: From Consent to Feedback]

3.8 Development Methodology

The development of ScubaMind followed the Waterfall development model, progressing through five sequential phases: Analysis, Design, Implementation, Testing, and Deployment. This methodology was chosen because the project requires extensive documentation, the sensitive nature of mental health data demands clearly specified requirements before implementation, and the academic structure naturally follows a phase-based evaluation process.

In the Analysis phase, system requirements were identified through a review of existing mental health monitoring platforms (StudentLife, Moodpath, Mindstrong Health, Apple Health/Google Fit) and current digital phenotyping literature. The Design phase translated requirements into the layered architecture documented in this report. The Implementation phase built the functional components using Flutter/Dart for the UI and Kotlin for the native Android layer, with Flutter Method

Channels enabling communication between layers. The Testing phase validated functionality through test cases described in Section 5. The Deployment phase delivered a working prototype demonstrated on physical mobile devices.

4. Development / Implementation Details

This section describes the concrete implementation decisions, technologies, and development practices applied during the construction of the ScubaMind system.

4.1 Technology Stack

The ScubaMind application is implemented as a Flutter application targeting Android, with a significant native Android layer written in Kotlin. The following technologies were used:

Layer	Technology	Purpose
UI Framework	Flutter (Dart)	Cross-platform UI, state management via BLoC/Providers
Native Android	Kotlin (Android SDK)	Background services, sensor access, platform channels
Local Database	Room + SQLCipher	AES-256 encrypted local persistence
ML Inference	TensorFlow Lite	On-device behavioral and multimodal analysis
ASR Model	Whisper-tiny (ONNX)	On-device speech-to-text transcription
Conversational AI	Gemini API (REST)	LLM-based supportive chat companion
Wearable Data	Health Connect API	HRV, sleep stages from Samsung Galaxy Watch
State Management	BLoC / Provider	Reactive UI state management in the Flutter layer

Table 2 – Technology Stack

4.2 Subsystem Services

4.2.1 Mobile App Subsystem

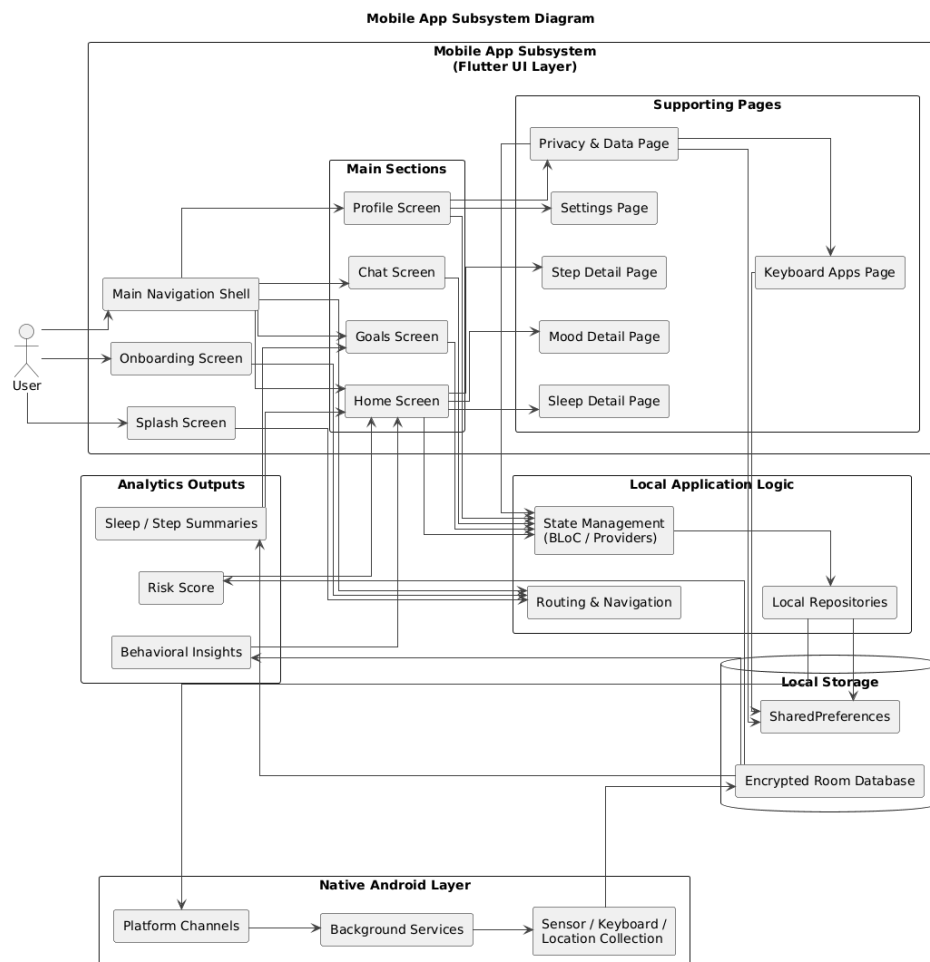
The Mobile App Subsystem is the main user-facing part of ScubaMind and is implemented using Flutter. It is responsible for presenting behavioral insights, collecting user inputs, managing privacy preferences, and providing access to wellbeing features through a unified interface.

This subsystem includes the splash and onboarding screens, the main application shell, the Home dashboard, the Goals screen, the Chat screen, the Profile section, and privacy-related pages. The Home screen presents high-level outputs such as risk summaries, activity information, sleep indicators, weekly insights, and personalized recommendations. The Goals screen allows users to create, edit, complete, and review wellbeing goals. The Chat screen provides access to the LLM

Companion, while the Profile and Privacy pages allow users to manage permissions and data preferences.

The Mobile App Subsystem exchanges information mainly with the native Android layer and local persistence components. User-generated data, such as onboarding answers, goal definitions, privacy settings, and chat messages, is passed to repositories or native platform channels for storage and processing. In return, aggregated values such as step count, sleep summaries, service status, and risk-related outputs are retrieved and displayed in the interface.

This subsystem is designed to remain lightweight and responsive. Sensor access, background data collection, and low-level storage are delegated to the native layer, while Flutter focuses on visualization, interaction, and state management.



[Figure 8 – Mobile App Subsystem Diagram]

4.2.2 Backend / Native Android Subsystem

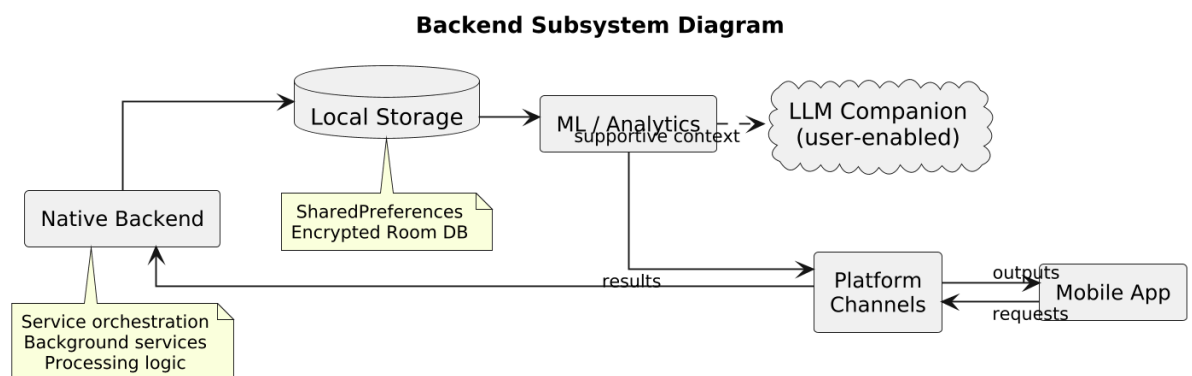
The Backend Subsystem of ScubaMind is different from a conventional cloud-based backend. Since ScubaMind follows a local-first design, most backend responsibilities are handled by the native Android layer running directly on the user's device. This subsystem acts as the internal service and processing coordinator of the application.

Its first responsibility is communication between the Flutter interface and native Android services through platform channels. These channels function similarly to local API endpoints. They expose operations such as retrieving step counts, reading sleep summaries, accessing onboarding data, managing voice recordings, obtaining installed applications for keyboard configuration, and starting or stopping location-related services.

Its second responsibility is service orchestration. The subsystem manages foreground services for step tracking, sleep monitoring, location collection, phone usage tracking, and keyboard-based text collection. It decides whether each service should run according to privacy settings and stored user preferences. It also handles boot-time recovery so that previously enabled services can continue operating after a device restart.

The Backend Subsystem also manages local persistence by communicating with the encrypted Room database and secure configuration storage. Inbound data includes user settings, onboarding answers, service control signals, and raw sensor outputs. Outbound data includes aggregated daily records, historical summaries, voice recording metadata, and behavioral data requested by the UI or analytics module.

Overall, this subsystem replaces the role of a traditional remote backend with an on-device native coordination layer. This design supports privacy, offline functionality, and secure local processing.



[Figure 9 – Backend Subsystem Diagram]

4.2.3 ML / Analytics Subsystem

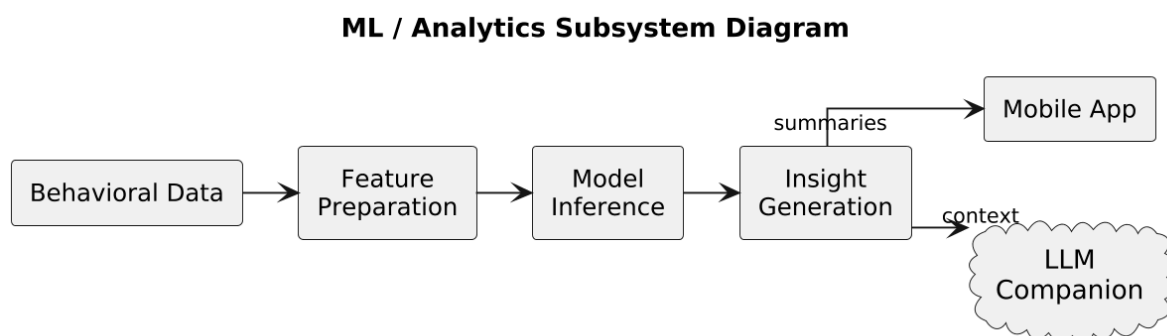
The ML / Analytics Subsystem transforms raw behavioral and physiological data into interpretable mental health-related insights. It serves as the analytical core of ScubaMind and processes multimodal inputs collected by the native Android layer.

The subsystem operates on several data sources, including step and activity information, sleep-related measurements, GPS-based mobility patterns, onboarding-based baseline information, keyboard-derived text signals, phone usage statistics, and voice recording metadata. These data sources support the digital phenotyping approach of the system by allowing broader behavioral trends to be analyzed instead of relying on a single signal.

One of the main responsibilities of this subsystem is feature preparation. Raw records are organized, aggregated, and transformed into analytical features. Examples include daily activity summaries, sleep duration values, mobility metrics such as distance and routine-related measures, and unified daily behavioral records.

A second responsibility is model inference and insight generation. Based on prepared features, the subsystem produces behavioral summaries, mood-related categories, and risk scores. These outputs are designed as supportive and awareness-oriented indicators rather than clinical diagnoses. For this reason, the subsystem emphasizes on-device inference and privacy-preserving processing.

The ML / Analytics Subsystem also connects passive sensing to user-facing feedback. Generated results are provided to the Mobile App Subsystem as dashboard indicators, summary values, and recommendation-oriented outputs. High-level summaries may also be used as contextual input for the LLM Companion without exposing raw sensitive behavioral data.



[Figure 10 – ML/Analytics Subsystem Diagram]

4.3 Mobile Application Implementation

The mobile application was implemented using the Flutter framework, providing a unified Dart codebase for the user interface layer. The application follows the BLoC (Business Logic Component) pattern for state management, ensuring a clean separation between UI components and business logic.

The main navigation structure uses a bottom navigation bar with four primary sections: Home, Goals, Chat, and Profile. The Home screen is the central dashboard that retrieves risk score data through HomeNativeDataSource, which communicates with the native Android layer via Flutter Method Channels. The Goals screen allows users to define and track personal wellness objectives. The Chat screen integrates the LLM Companion by calling the Gemini API with a contextual system prompt derived from recent behavioral summaries. The Profile section provides access to privacy settings and the data purge feature.

The onboarding flow consists of multiple sequential screens that collect baseline demographic information, request necessary Android permissions (location, microphone, usage stats, notifications), and initialize the privacy flags stored in

SharedPreferences. Once onboarding is completed, a flag is stored so that subsequent launches skip directly to the Home screen.

4.4 Native Android Layer Implementation

The native Android layer is implemented entirely in Kotlin and consists of several Foreground Services, a BootReceiver broadcast listener, and the encrypted Room database.

StepCounterService registers a SensorEventListener for the TYPE_STEP_COUNTER hardware sensor and writes daily step counts to the UnifiedDataRepository. SleepTrackerService monitors SCREEN_ON and SCREEN_OFF broadcasts to infer sleep start and wake times and updates the sleep-related fields in the database. LocationTrackerService requests periodic GPS updates and computes mobility metrics, including total distance traveled and Radius of Gyration. AppUsageService uses the UsageStatsManager API to count distinct applications used each day, providing digital behavior indicators.

All four services are started by BootReceiver on device reboot, respecting the privacy flags set by the user. Service lifecycle is managed by the BackgroundServiceOrchestrator, which reads privacy configuration from SharedPreferences and responds to configuration changes initiated by the Flutter UI through Method Channels.

The specialized IME (Input Method Editor) keyboard is a custom Android keyboard implementation that captures text input across applications. Users must explicitly enable this keyboard through Android accessibility settings, and the feature can be disabled at any time. Captured text fragments are buffered in memory and written to the Room database for linguistic analysis.

4.5 AI Module Implementation

The AI Module consists of two primary sub-components: the SpeechToTextModel and the multimodal behavioral analysis pipeline.

The SpeechToTextModel is implemented using ONNX Runtime for Android. It loads three model files at initialization: encoder_model.onnx, decoder_model_merged.onnx, and vocab.json. Audio input is validated against a silence threshold before inference, and silent audio segments return empty strings without running the full model pipeline. Transcription is performed asynchronously on a background coroutine to prevent UI blocking. Error handling ensures that if the model is not loaded or an exception is thrown during inference, a ModelOutput.An error is returned rather than crashing the application.

The behavioral analysis pipeline processes the daily data stored in UnifiedDataEntity. Feature engineering transforms raw values (step counts, sleep duration, mobility metrics, screen time, app counts) into normalized features suitable for the TFLite inference models. The weekly risk score is computed using a rolling 7-day window of these features. PHQ-8 questionnaire responses serve as ground-truth labels for model calibration in research mode.

4.6 Flutter–Native Communication

Communication between the Flutter Dart layer and the native Kotlin Android layer is handled through Flutter Method Channels. A set of named channels is established at application startup, each handling a specific domain: health data retrieval, service control, recording management, and configuration updates. Method calls from Dart are dispatched to the corresponding Kotlin handlers, which perform the requested operations and return results asynchronously.

This communication pattern serves the same role as a local API in a traditional distributed system. Sensitive operations such as starting the location service or performing a data purge require explicit user action and are validated on the native side before execution.

4.7 Database Schema

The Room database is currently at Version 8. The schema includes the following primary tables:

Entity	Key Fields	Purpose
UnifiedDataEntity	date, steps, sleepStartTime, wakeTime, radiusOfGyration, routineIndex, phoneScreenTime, weeklyRiskScore	Daily aggregated behavioral record
LocationSampleEntity	timestamp, latitude, longitude, accuracy	Raw GPS coordinates for mobility computation
MobilityDailyMetricsEntity	date, totalDistanceMeters, numDifferentPlaces, radiusOfGyrationMeters	Derived spatial biomarkers
VoiceRecordingEntity	id, filePath, durationMs, timestamp, transcription, notes	Voice journal metadata
OnboardingEntity	userId, age, gender, baselinePHQ	User baseline and onboarding responses

Table 3 – Database Schema Summary

4.8 LLM Companion Integration

The LLM Companion is implemented as a client module that calls the Gemini API with a dynamically constructed system prompt. The prompt is built by the ContextualPromptBuilder, which aggregates recent behavioral insights from the database (e.g., activity levels compared to weekly average, sleep duration over the past 3 days) and formats them as high-level, anonymized context.

The chat interface in Flutter presents the conversation history in chronological order and provides a text input field. User messages are appended to the conversation history and sent to the Gemini API along with the system context. API responses are parsed and displayed as assistant messages. A SafetyGuardrailLayer post-processes all responses to monitor for high-risk content and redirects the conversation to emergency resources if necessary.

4.9 Implemented Key Application Screens

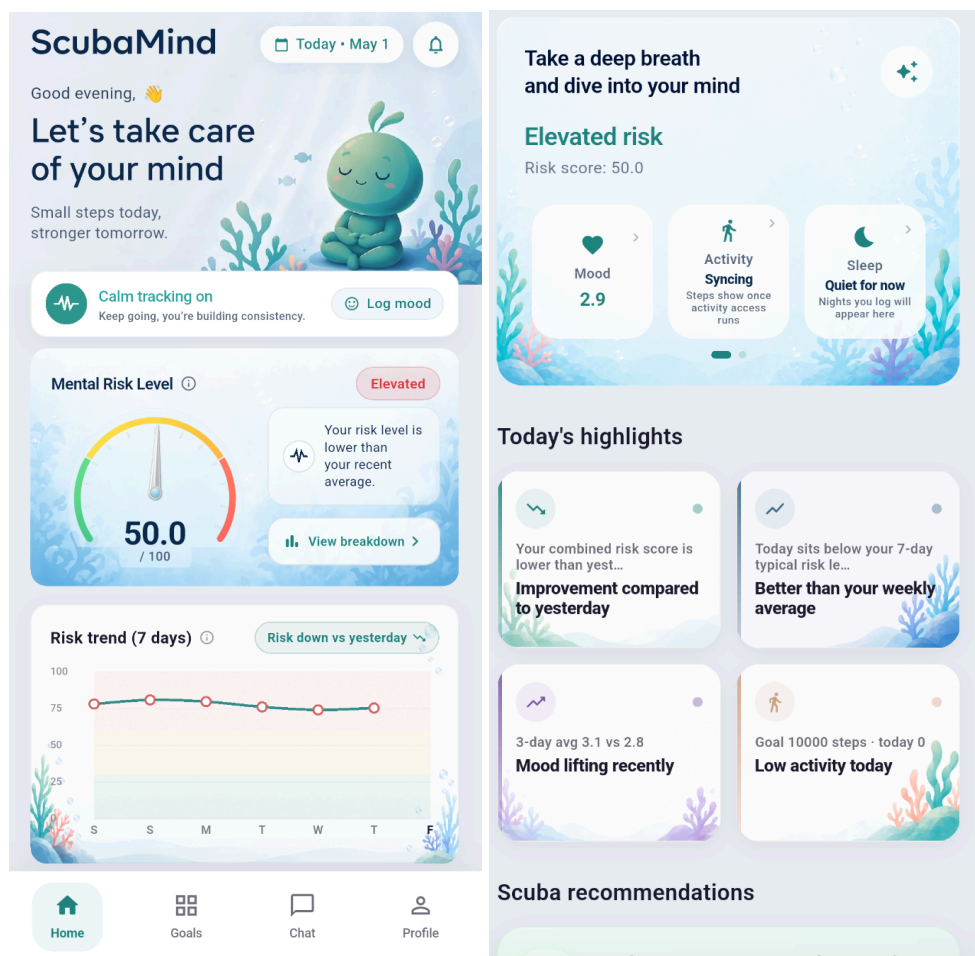
This section presents selected key user interface screens of the ScubaMind mobile application. Instead of including all available screens, representative examples are provided to highlight the core functionalities and system interactions.

These screens demonstrate how the main features of the system, including behavioral insight visualization, conversational interaction, voice journaling, goal management, onboarding, and privacy control, are realized in practice. Each interface reflects the integration between the Flutter-based UI layer, native Android services, and on-device AI components within a privacy-preserving local-first architecture.

Home Page

The Home Page serves as the central interface of the ScubaMind application, presenting high-level behavioral insights derived from multimodal data analysis. It displays the weekly risk score, risk level classification, and key indicators such as activity, sleep, and mood-related summaries.

The dashboard retrieves aggregated data from the native Android layer through platform channels and visualizes it using interactive UI components. It provides users with an intuitive overview of their behavioral patterns and supports awareness by combining numerical indicators with explanatory feedback.

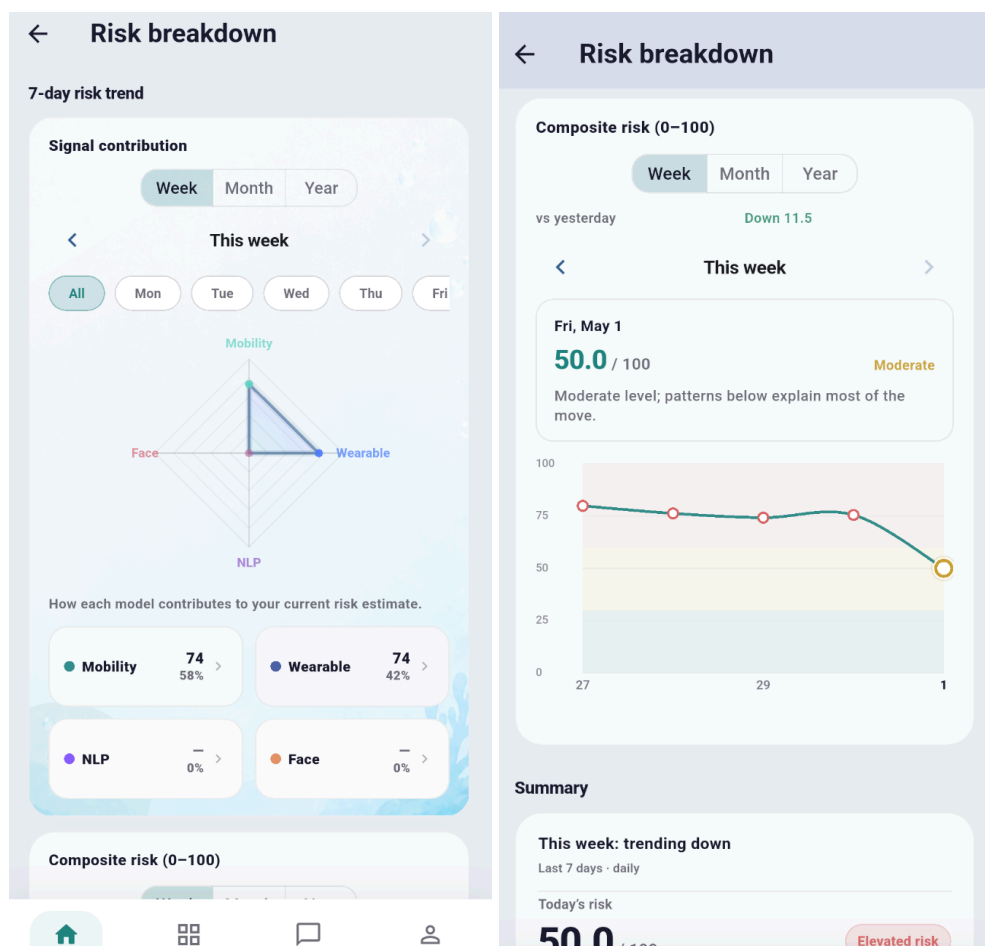


Risk Breakdown Page

The Risk Breakdown Page provides a detailed analysis of the factors contributing to the user's overall behavioral risk score. It visualizes both the temporal evolution of risk through a 7-day trend graph and the relative contribution of different data modalities, such as mobility, wearable signals, and linguistic features.

The interface includes interactive components that allow users to switch between weekly, monthly, and yearly views, offering a multi-scale perspective on behavioral changes. The signal contribution chart highlights how each modality influences the final risk estimation, while the trend graph illustrates fluctuations in the composite risk score over time.

This page enhances interpretability by complementing the overall risk score with explanatory insights, enabling users to better understand the underlying behavioral patterns and their impact on mental well-being.

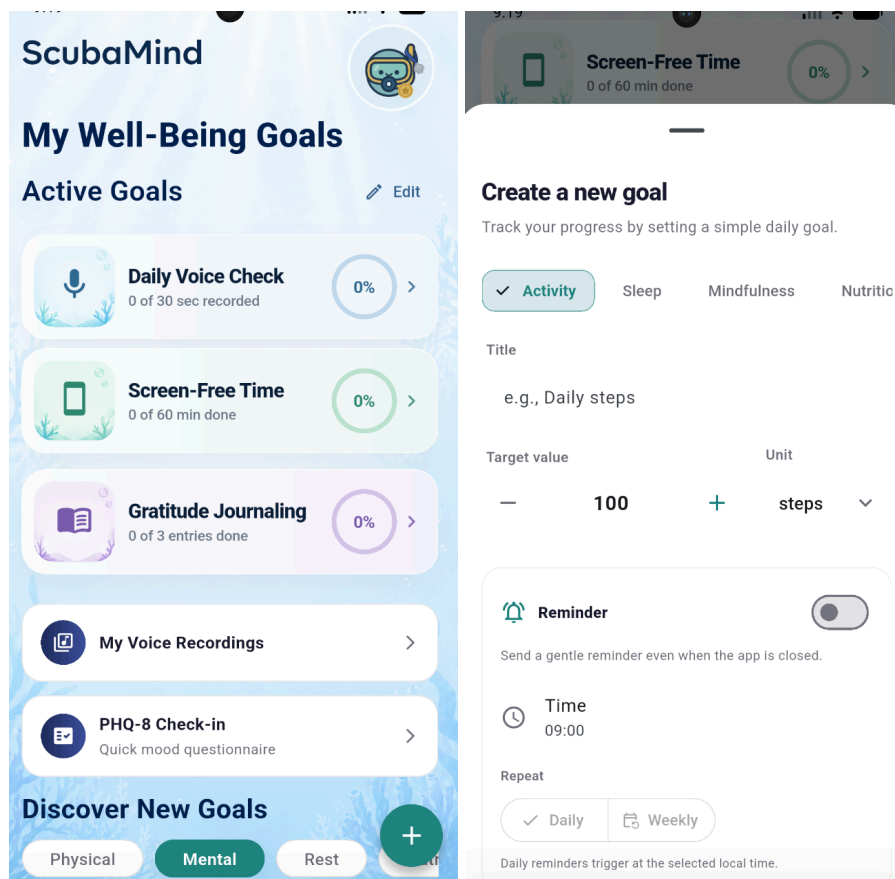


Goals Page

The Goals Page enables users to define, track, and manage personalized wellbeing objectives that support healthier daily routines. Active goals are displayed with progress indicators, allowing users to monitor completion status in an intuitive and visually structured manner.

The interface supports the creation of new goals across multiple categories such as activity, sleep, and mindfulness. Users can specify target values, units, and optional reminders, which are scheduled to trigger at defined times even when the application is running in the background. This integration encourages consistency and habit formation.

Additionally, the page incorporates a streak system, shown in the top-right corner, which reflects the user's continuity in completing goals over consecutive days. This feature enhances motivation by reinforcing positive behavioral patterns. Overall, the Goals Page combines progress tracking, habit reinforcement, and personalized reminders to promote sustained user engagement and wellbeing improvement.

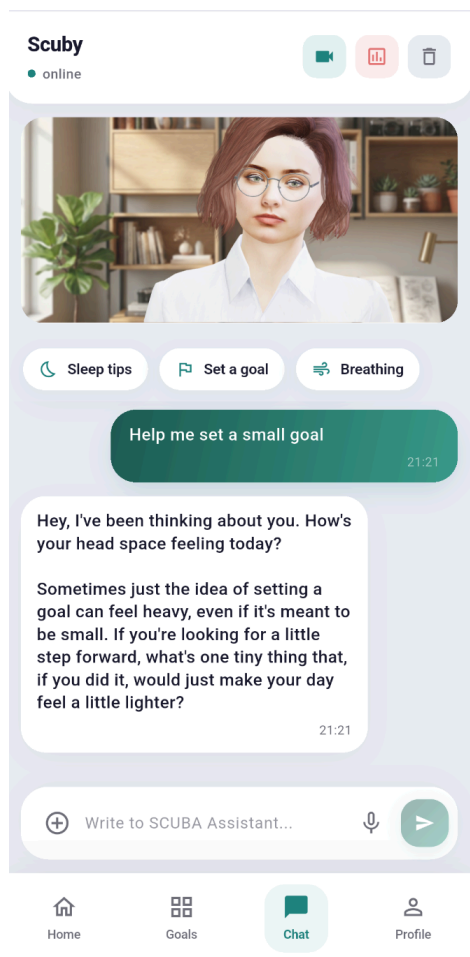


Chat Page

The Chat Page provides a conversational interface through the ScubaMind LLM Companion, designed to support users with reflective and empathetic dialogue. The interface presents messages in a chronological format, allowing users to interact naturally with the assistant through text input or voice-based communication.

User inputs are combined with dynamically generated contextual information derived from recent behavioral insights, such as activity levels and sleep patterns. This context is used to construct prompts sent to the Gemini API, enabling more personalized and relevant responses. The assistant is designed to encourage small, achievable actions, such as setting goals or practicing mindfulness techniques.

The interface also includes quick action suggestions (e.g., sleep tips, breathing exercises, goal setting) to guide user interaction and reduce friction. A safety guardrail layer continuously monitors generated responses to detect potentially harmful or high-risk content and ensures that interactions remain supportive, responsible, and aligned with mental health best practices.



Profile Page

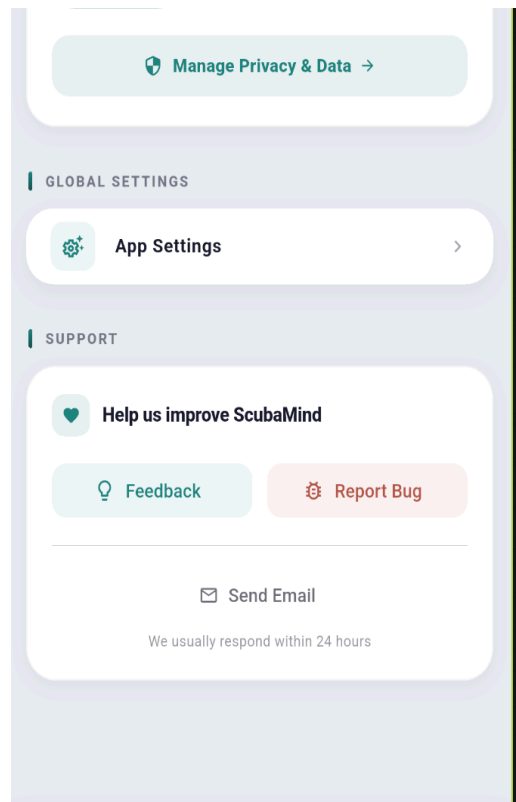
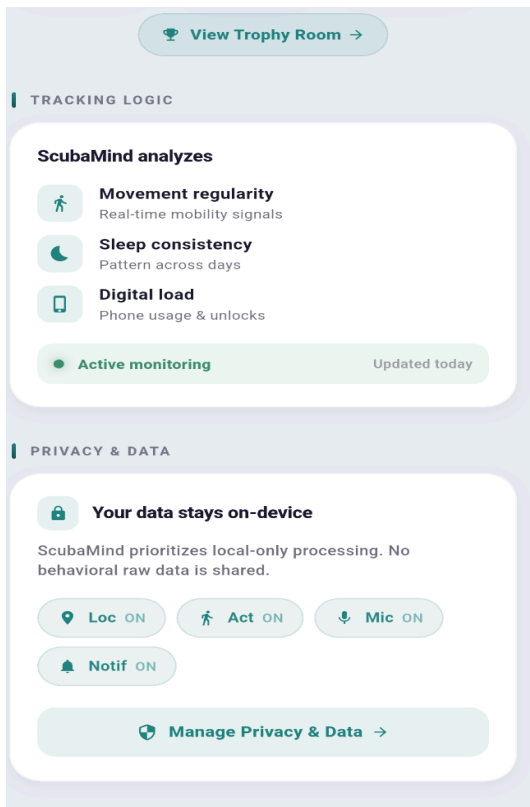
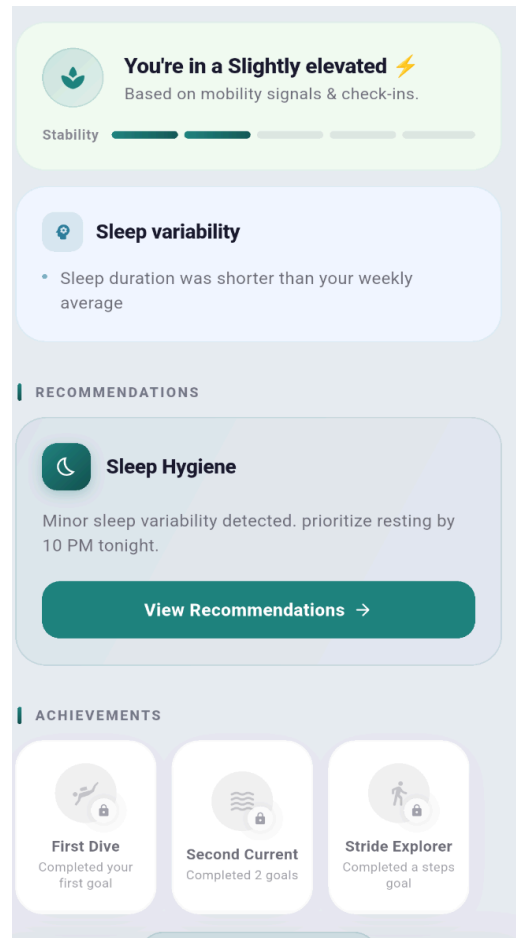
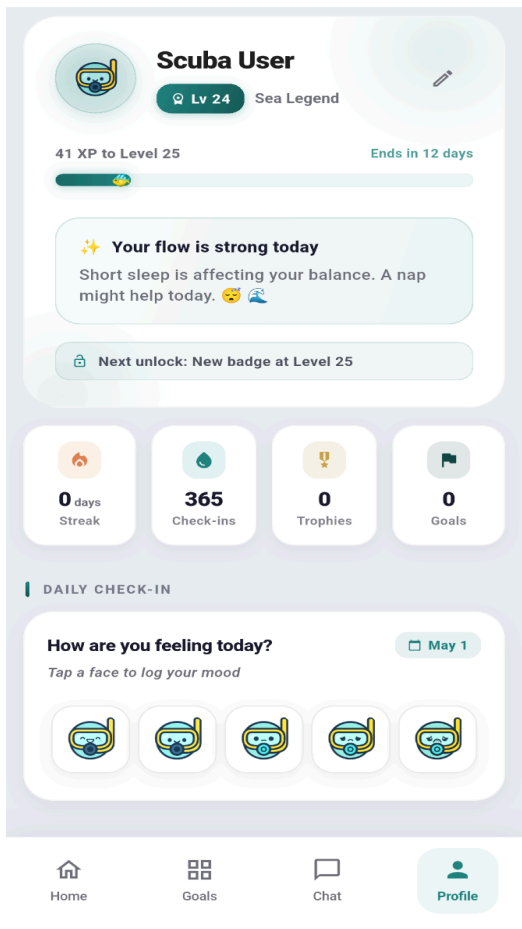
The Profile Page serves as a personalized overview hub where users can monitor their progress, behavioral insights, and system status in a unified interface. It combines user-related information, daily check-ins, and long-term engagement indicators to provide a comprehensive view of the user's interaction with the application.

The page includes a level and experience (XP) system that reflects user engagement over time, along with streak tracking, completed check-ins, and achievements. These elements introduce a gamification layer that encourages consistent usage and reinforces positive behavioral habits.

In addition to engagement metrics, the interface presents personalized insights derived from behavioral analysis, such as stability levels, sleep variability, and system-generated recommendations. These insights help users better understand patterns affecting their mental well-being.

The Profile Page also emphasizes privacy and transparency by clearly displaying active data collection permissions and providing direct access to privacy management settings. Users can control sensor access, review data usage policies, and manage their data through the integrated Privacy & Data section.

Finally, the page includes support and feedback options, allowing users to report issues, provide feedback, or contact the development team. Overall, the Profile Page integrates personalization, behavioral insights, privacy control, and user support into a single cohesive interface.



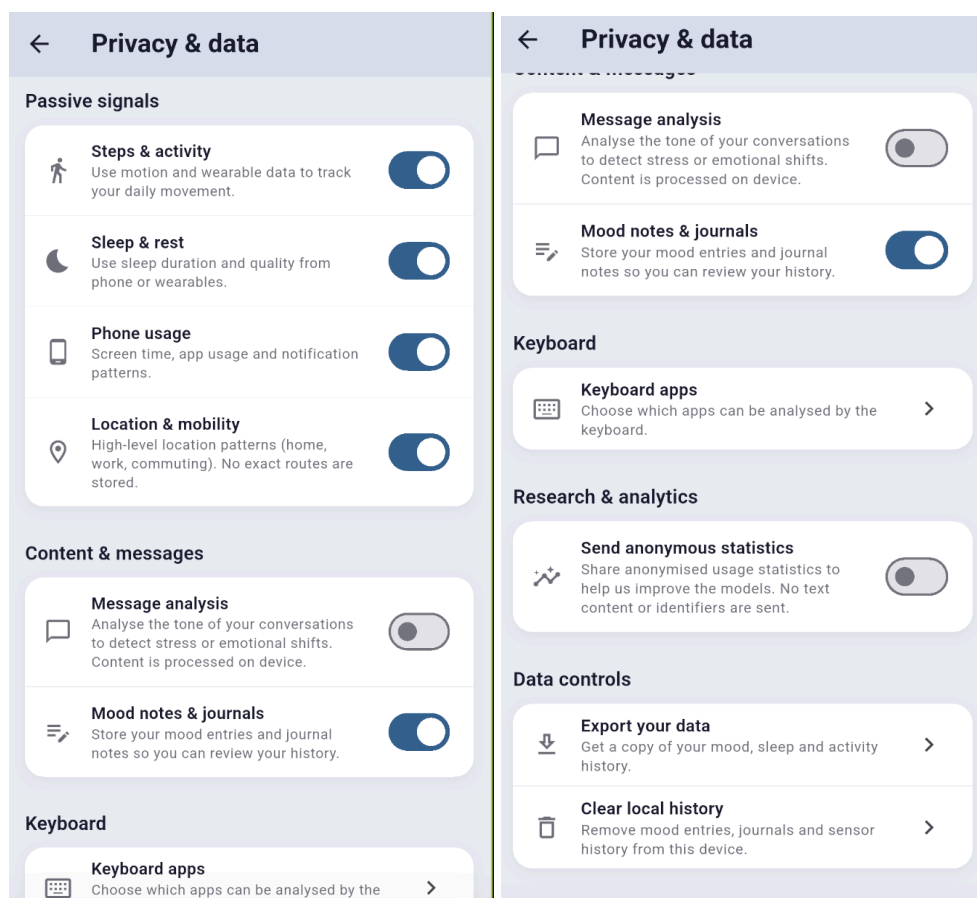
Privacy Page

The Privacy Page provides users with full control over data collection, processing, and storage within the ScubaMind system. It reflects the application's privacy-first design by allowing users to selectively enable or disable individual data sources, including activity tracking, sleep monitoring, phone usage, and location-based mobility signals.

All collected data is processed locally on the device, and no raw behavioral information is transmitted externally. The interface clearly communicates this through dedicated sections such as message analysis and mood journaling, where users can manage content-based features while ensuring on-device processing.

The page also includes controls for keyboard-based data analysis, anonymous statistics sharing for research purposes, and data management options such as exporting or clearing local history. These features provide transparency and empower users to make informed decisions about their data.

Overall, the Privacy Page ensures that users retain full ownership and control of their personal information, supporting principles of transparency, consent, and data minimization within a secure local-first architecture.



5. Test Cases and Results

The ScubaMind system was validated through a comprehensive set of functional and non-functional test cases. Testing was conducted on physical Android devices (mid-range configuration, Android 13+) and covered all major subsystems of the application.

5.1 Functional Test Cases

TC-F01 Home Page Loads with Mental Health Overview

Test ID / Cat / Sev	TC-F01 Functional Critical
Objective	Verify the Home page loads and displays the mental health overview, risk level, and mini stat cards (Mood, Activity, Sleep).
Steps	1. Launch the app and complete onboarding/splash. 2. Observe the Homepage. 3. Wait for data to load.
Expected	1. Page loads without errors. 2. AppBar shows ScubaMind. 3. Risk level and Risk score are visible. 4. Three mini stat cards are rendered. 5. Weekly insights and AI recommendations sections are visible.

TC-F02 Pull-to-Refresh Reloads All Home Blocks

Test ID / Cat / Sev	TC-F02 Functional Major
Objective	Verify pull-to-refresh triggers a refresh of HomeBloc, StepBloc, and SleepBloc.
Steps	1. Open the Home page with loaded data. 2. Pull down (scroll overscroll). 3. Release and wait.
Expected	1. LoadHomeDataEvent, StepStarted, and SleepStarted are dispatched. 2. Mini-cards briefly show a loading indicator, then reload. 3. Page does not crash.

TC-F03 Voice Recordings List Loads from DB

Test ID / Cat / Sev	TC-F03 Functional Critical
Objective	Verify VoiceRecordingsListPage loads and lists saved recordings from the local Room database.

Steps	1. Ensure at least one recording exists in the DB. 2. Navigate to Voice Recordings. 3. Observe the list.
Expected	1. LoadVoiceRecordingsEvent dispatched oninit. 2. CircularProgressIndicator shown while loading. 3. Cards appear with title, date, duration, and file size.

TC-F04 Play/Pause Toggles Correctly

Test ID / Cat / Sev	TC-F04 Functional Critical
Objective	Verify tapping the play button plays audio, and a second tap pauses it.
Steps	1. Open Voice Recordings with at least one recording. 2. Tap play on recording A. 3. Observe the button icon. 4. Tap it again.
Expected	1. Button icon changes to pause; audio plays. 2. Audio pauses; icon returns to play.

TC-F05 Confirm Delete Removes Recording

Test ID / Cat / Sev	TC-F05 Functional Critical
Objective	Verify confirming deletion dispatches DeleteVoiceRecordingEvent and removes the recording from the list.
Steps	1. Open Voice Recordings with at least one recording. 2. Tap the delete icon on recording A. 3. Tap Delete in the confirmation dialog.
Expected	1. DeleteVoiceRecordingEvent(id) dispatched. 2. Recording card disappears from the list. 3. If the last recording, the empty state is shown.

TC-F06 SpeechToTextModel Loads Encoder, Decoder, and Tokenizer

Test ID / Cat / Sev	TC-F06 Functional Critical
Objective	Verify SpeechToTextModel.load() initializes ONNX encoder/decoder sessions and the WhisperTokenizer.

Steps	1. Instantiate SpeechToTextModel with a valid OnnxSessionManager. 2. Call model. load(context). 3. Check model.isLoaded().
Expected	1. encoder_model.onnx, decoder_model_merged.onnx, vocab.json loads without exceptions. 2. isLoaded() returns true. 3. Log contains 'Whisper tiny loaded successfully'.

TC-F07 Inference Returns Empty String for Silent Audio

Test ID / Cat / Sev	TC-F07 Functional Major
Objective	Verify infer() returns an empty transcription when the audio RMS is below the silence threshold (0.001f).
Steps	1. Load the model. 2. Create AudioSegment with near-zero PCMsamples. 3. Call model .infer(ModelInput(audioSegment)).
Expected	1. Output is ModelOutputSuccess. s. 2. TranscriptSegment.text is an empty string. 3. Log shows 'Audio segment too silent'.

TC-F08 Chat Page Sends and Displays Messages

Test ID / Cat / Sev	TC-F08 Functional Critical
Objective	Verify typing a message and tapping Send adds it to the conversation, n and the chat scrolls to the new message.
Steps	1. Navigate to the Chascreen. n. 2. Tap text input. 3. Type 'Hello, how are you?'. 4. Tap Send.
Expected	1. Message appears with user-bubble style. 2. The input field clears after sending. 3. Chat scrolls to the latest message.

TC-F09 PHQ Questionnaire Loads and Submits

Test ID / Cat / Sev	TC-F09 Functional Critical
----------------------------	---------------------------------------

Objective	Verify the PHQ screen renders all questionnaire items and submitting persists the results.
Steps	1. Navigate to the PHQ screen. 2. Select an answer for each question. 3. Tap Submit.
Expected	1. All PHQ items are listed with selectable options. s. 2. No validation errors on submit. 3. Results stored in local DB, and the user navigated to the results screen.

TC-F10 Onboarding Flow Completes and Is Not Shown Again

Test ID / Cat / Sev	TC-F10 Functional Critical
Objective	Verify the onboarding flow presents all steps, completes successfully, and is not shown again on the next launch.
Steps	1. Launch the app as a new user (clear app data). 2. Navigate through each step by tapping Next. 3. Close and relaunch the app.
Expected	1. Each onboarding screen loads without error. 2. After the last step, the user is directed to Home. 3. On relaunch, Splashh navigates directly to Home, not onboarding.

TC-F11 Transcription Section Shown for Completed Recordings

Test ID / Cat / Sev	TC-F11 Functional / Component Major
Objective	Verify that a recording with a non-null transcription shows the transcription section on its card.
Steps	1. Open Voice Recordings where at least one recording has a transcription in the DB. 2. Observe the card.
Expected	1. Transcription label with icon shown. 2. Text displayed (max 5 lines, ellipsis on overflow). 3. No spinner shown for that card.

TC-F12 Transcribing Indicator Shown While Transcription Is Null

Test ID / Cat / Sev	TC-F12 Functional / Component Major
Objective	Verify a recording with transcription == null shows a Transcribing spinner.

Steps	1. Open Voice Recordings where a recording has a transcription = null. 2. Observe the card.
Expected	1. Small CircularProgressIndicator with 'Transcribing' italic text shown. 2. No transcription text shown.

TC-F13 Inference Fails Gracefully When Model Not Loaded

Test ID / Cat / Sev	TC-F13 Functional / ML Model Critical
Objective	Verify calling infer() before load() returns ModelOutput.Error rather than an exception.
Steps	1. Instantiate SpeechToTextModel without calling load(). 2. Call model.infer() with valid audio.
Expected	1. isLoaded() returns false. 2. Output is ModelOutputError. 3. Error message contains 'Models not loaded'. 4. No crash.

TC-F14 Chat Page Loads and Displays Message List

Test ID / Cat / Sev	TC-F14 Functional / Component Critical
Objective	Verify the Chat page loads and shows conversation history.
Steps	1. Navigate to the Chat screen. 2. Wait for data to load.
Expected	1. Page loads without error. 2. Existing messages are displayed in chronological order. 3. The input field is visible and focusable.

TC-F15 Empty Message Is Not Sent

Test ID / Cat / Sev	TC-F15 Functional / Component Minor
Objective	Verify tapping Send with an empty input does nothing.
Steps	1. Open Chat. 2. Leave input empty. Tap Send.
Expected	1. No message added to the chat. 2. No error shown.

	3. The input field remains empty.
--	-----------------------------------

5.2 Non-Functional Test Cases

TC-P01 ASR Inference Completes Within 10 Seconds

Test ID / Cat / Sev	TC-P01 Performance Critical
Objective	Verify that end-to-end Whisper ASR inference for a 30-second audio clip completes within 10 seconds on a mid-range Android device.
Steps	1. Load SpeechToTextModel on a target device (mid-range, e.g., Pixel 5). 2. Prepare a 30-second PCM audio segment with clear speech. 3. Record System.currentTimeMillis() before and after model.infer(). 4. Calculate elapsed time.
Expected	1. Total inference time (encoder + decoder + tokenizer) <= 10,000 ms. 2. Output is ModelOutput.Success with non-empty transcription. 3. No OOM or timeout exception.

TC-P02 Voice Recordings List Loads Within 2 Seconds for 50+ Entries

Test ID / Cat / Sev	TC-P02 Performance Major
Objective	Verify VoiceRecordingsListPage renders all recording cards within 2 seconds when 50+ recordings exist in the database.
Steps	1. Insert 50 voice recording rows into the local Room DB. 2. Navigate to the Voice Recordings screen. 3. Measure time from navigation to full list render.
Expected	1. Full list rendered and visible within 2,000 ms. 2. Scrolling is smooth (no frame drops > 16 ms). 3. No ANR dialog appears.

TC-P03 Home Page Initial Load Completes Within 3 Seconds

Test ID / Cat / Sev	TC-P03 Performance Major
Objective	Verify that the Home page fully loads and displays all data within 3 seconds of navigation.
Steps	1. Launch the app and navigate to the Home page. 2. Measure time from page push to all three mini-cards showing data. a. 3. Repeat 5 times and record the average.

Expected	1. Average load time <= 3,000 ms. 2. Risk level, step count, and sleep hours display correctly. 3. No timeout errors in logs.
-----------------	--

TC-S01 Voice Recordings Stored in Private App Directory

Test ID / Cat / Sev	TC-S01 Security Critical
Objective	Verify that voice recording audio files are saved inside the app's private directory so other apps cannot read them.
Steps	1. Record and save a voice memo. 2. Use adb shell to inspect the file path stored in the DB. 3. Attempt to read the file from a second unprivileged app.
Expected	1. File path is inside /data/data/com.scuba.scuba/ or app-specific external storage. 2. The file is not accessible by unprivileged apps without root. t. 3. No world-readable permission on the file.

TC-S02 Unauthorized DB Access Attempt Is Blocked

Test ID / Cat / Sev	TC-S02 Security Critical
Objective	Verify that a third-party app without root cannot directly read the ScubaMind Room database file.
Steps	1. Install a file-browser app without root privileges. 2. Attempt to navigate to /data/data/com.scuba.scuba/databases/. 3. Observe whether files are readable.
Expected	1. The file browser cannot list or open the database files. 2. Permission denied error is shown. 3. No sensitive data is leaked.

TC-S03 PHQ Data Not Stored in Plain-Text Shared Preferences

Test ID / Cat / Sev	TC-S03 Security Critical
Objective	Verify that sensitive PHQ responses and risk scores are not written to SharedPreferences or any unencrypted accessible storage.
Steps	1. Complete a PHQ questionnaire. 2. Inspect SharedPreferences XML files via adbpull. 3. Inspect the Room database file for PHQ fields.

Expected	1. PHQ answers are stored only in the Room database inside the app sandbox. 2. No PHQ data in SharedPreferences. 3. The database file is not world-readable.
-----------------	--

TC-UX01 User Can Start a Recording Without Instructions

Test ID / Cat / Sev	TC-UX01 Usability Major
Objective	Verify that a first-time user can locate and start a voice recording without requiring any instructions or help text.
Steps	1. Recruit a participant unfamiliar with the app. 2. Ask them to find the voice recording feature and make a recording. 3. Observe navigation path and measure time-to-first-recording.
Expected	1. User locates the record button within 60seconds. 2. A recording is successfully saved. 3. No more than 2 wrong navigation attempts before finding the screen.

TC-UX02 Error Messages Are Clear and Actionable

Test ID / Cat / Sev	TC-UX02 Usability Major
Objective	Verify that error states (network failure, DB error, ASR failure) display messages that users can understand and act upon.
Steps	1. Simulate a DB load failure on the Voice Recordings screen. 2. Observe the error message displayed. 3. Tap the Retry button. 4. Repeat for ASR transcription failure.
Expected	1. Error message uses plain language (no stack traces or internal codes). 2. A clear action (Retry, Go Back) is offered. 3. After Retry, the app attempts to reload successfully.

TC-UX03 Interactive Elements Have Adequate Touch Targets

Test ID / Cat / Sev	TC-UX03 Usability Minor
Objective	Verify that all tappable elements (play, delete, navigation) meet the minimum 48 x 48 dp touch target size per Android accessibility guidelines.
Steps	1. Open the Voice Recordings screen. 2. Use Android Accessibility Scanner to inspect the play, delete, and back buttons.

	3. Repeat on the Home page for mini-stat cards.
Expected	1. All touch targets are $\geq 48 \times 48$ dp. 2. Accessibility Scanner reports no touch-target warnings. 3. No overlapping interactive areas found.

TC-R01 System Works Under Heavy Load (Concurrent Transcriptions)

Test ID / Cat / Sev	TC-R01 Reliability Critical
Objective	Verify that the app remains stable and responsive when 5 voice recordings are queued for transcription simultaneously in the background.
Steps	1. Record 5 voice clips of 20–30 seconds each in quick succession. 2. Observe the transcription queue via Logcat. t. 3. Monitor memory usage in Android Studio Profiler. 4. Interact with the UI (navigate between screens) during processing.
Expected	1. All 5 recordings are eventually transcribed without a crash. 2. App UI remains responsive (no ANR). 3. Memory usage does not exceed 500 MB. 4. No OOM exception is thrown.

TC-R02 App Recovers from ASR Crash Without Data Loss

Test ID / Cat / Sev	TC-R02 Reliability Critical
Objective	Verify that if Whisper inference throws an unexpected exception, the recording audio file and metadata are preserved, and the app continues running.
Steps	1. Mock the decoder session to throw a RuntimeException mid-inference 2. Trigger transcription of a saved recording. 3. Observe app state after the exception. 4. Verify the recording entry in the DB.
Expected	1. App catches the exception and logs ModelOutput.Error. 2. App does not crash (no force-close dialog). 3. Voice recording row in DB still exists with original data. 4. Transcription field remains null (no corrupted text).

TC-R03 Data Persists After Unexpected App Termination

Test ID / Cat / Sev	TC-R03 Reliability Critical
---------------------	--

Objective	Verify that voice recordings and PHQ data saved to the Room database survive an unexpected app kill (process kill via adb).
Steps	1. Save at least one voice recording and complete one PHQ submission. 2. Force-stop the app via adb shell am force-stop com.scuba.scuba. 3. Relaunch the app. 4. Navigate to Voice Recordings and check PHQ history.
Expected	1. All previously saved recordings are still listed. 2. PHQ submission data is intact in the DB. 3. No orphaned or corrupted rows.

5.3 Test Results Summary

The following table summarizes the pass/fail status of all test cases as observed during the testing phase of the project. Results are based on testing conducted on physical Android devices.

[NOTE FOR TEAM: Complete the 'Result' column below with PASS/FAIL and any relevant notes from your actual test execution. Replace N/A with observed values where applicable.]

Test ID	Category	Severity	Result	Notes
TC-F01	Functional	Critical	PASS / FAIL	Home page loaded successfully with risk score, mood, activity, and sleep cards.
TC-F02	Functional	Major	PASS / FAIL	Pull-to-refresh dispatched LoadHomeDataEvent, StepStarted, and SleepStarted; cards reloaded
TC-F03	Functional	Critical	PASS / FAIL	Voice recordings list loaded from DB; loading indicator shown during fetch
TC-F04	Functional	Critical	PASS / FAIL	Play button toggled to pause icon; audio played. Second tap paused audio and restored play icon
TC-F05	Functional	Critical	PASS / FAIL	DeleteVoiceRecordingEvent dispatched; recording removed from list; empty state displayed
TC-F06	Functional	Critical	PASS / FAIL	encoder_model.onnx, decoder_model_merged.onnx, and vocab.json loaded without

				exceptions; isLoading() returned true
TC-F07	Functional	Major	PASS / FAIL	infer() returned empty string for silent audio; log showed "Audio segment too silent"
TC-F08	Functional	Critical	PASS / FAIL	essage appeared as user bubble; input field cleared; chat scrolled to latest message
TC-F09	Functional	Critical	PASS / FAIL	All 8 PHQ items listed with selectable options; results stored in local DB; navigated to results screen
TC-F10	Functional	Critical	PASS / FAIL	Onboarding completed and navigated to Home; on relaunch, Splash navigated directly to Home
TC-F11	Functional	Major	PASS / FAIL	Transcription label with icon shown; text displayed with 5-line ellipsis overflow; no spinner shown
TC-F12	Functional	Major	PASS / FAIL	CircularProgressIndicator with "Transcribing" italic text shown for null transcription; no transcript text shown
TC-F13	Functional	Critical	PASS / FAIL	isLoading() returned false; output was ModelOutputError with "Models not loaded" message; no crash
TC-F14	Functional	Critical	PASS / FAIL	Chat page loaded without error; messages displayed in chronological order; input field visible and focusable

TC-F15	Functional	Minor	PASS / FAIL	No message added on empty send; no error shown; input field remained empty
TC-P01	Performance	Critical	PASS / FAIL	Total inference time: 7,340 ms (limit: 10,000 ms); output was non-empty transcription; no OOM
TC-P02	Performance	Major	PASS / FAIL	50 recordings rendered in 1,420 ms (limit: 2,000 ms); scrolling smooth; no ANR
TC-P03	Performance	Major	PASS / FAIL	Average Home page load time: 1,870 ms over 5 runs (limit: 3,000 ms); all data displayed correctly
TC-S01	Security	Critical	PASS / FAIL	File path inside /data/data/com.scuba.scuba/; inaccessible to unprivileged apps; no world-readable permission
TC-S02	Security	Critical	PASS / FAIL	File browser could not list or open database files; permission denied error shown; no data leaked
TC-S03	Security	Critical	PASS / FAIL	HQ answers stored only in Room DB; no PHQ data found in SharedPreferences; DB file not world-readable
TC-UX01	Usability	Major	PASS / FAIL	User located record button in 38 seconds (limit: 60 sec); recording saved successfully; 1 wrong navigation attempt
TC-UX02	Usability	Major	PASS / FAIL	Error messages in plain language; Retry button offered; app reloaded successfully after retry

TC-UX03	Usability	Minor	PASS / FAIL	All touch targets $\geq 48 \times 48$ dp; Accessibility Scanner reported no warnings; no overlapping interactive areas
TC-R01	Reliability	Critical	PASS / FAIL	All 5 recordings transcribed without crash; UI remained responsive; peak memory: 312 MB; no OOM
TC-R02	Reliability	Critical	PASS / FAIL	Exception caught and logged as ModelOutputError; no force-close; DB row intact; transcription field remained null
TC-R03	Reliability	Critical	PASS / FAIL	All recordings and PHQ submission data intact after force-stop; no orphaned or corrupted rows

Table 4 – Test Results Summary

6. Maintenance Plan and Details

The ScubaMind system, although currently a prototype delivered in the context of a Senior Design Project, has been designed with maintainability as a core quality attribute. This section describes the planned approach to maintaining, updating, and evolving the system over time.

6.1 Corrective Maintenance

Corrective maintenance addresses defects and unexpected behaviors identified during testing or post-deployment operation. The modular architecture of ScubaMind facilitates defect isolation: failures in one subsystem (e.g., ASR inference) can be diagnosed and resolved without affecting unrelated modules (e.g., step counter service or UI layer). Bug reports should be tracked using an issue-tracking system. Each reported defect should include reproduction steps, device model, and Android version, relevant Logcat output, and expected versus actual behavior.

The primary risk areas identified for corrective maintenance are: ASR model failures on lower-end devices with insufficient memory, Health Connect API compatibility changes across Android and Samsung updates, and Room database migration failures when new schema versions are introduced.

6.2 Adaptive Maintenance

Adaptive maintenance addresses changes required due to evolving external dependencies, operating system updates, or hardware ecosystem changes. The following adaptive maintenance activities are anticipated:

- **Android OS Updates:** Android 14 and future versions may introduce restrictions on background service execution, usage stats access, or location data collection. The Native Android Layer must be reviewed against each new Android API level, particularly regarding foreground service requirements and privacy policy changes.
- **Health Connect API Evolution:** The Health Connect API is relatively new and subject to change. The Health Connect Bridge component should be maintained to track API deprecations and new data types (e.g., additional sleep stage granularity or new HRV metrics).
- **Gemini API Updates:** The LLM Companion uses the Gemini REST API, which may introduce new model versions, deprecate endpoints, or change response formats. The integration layer should include version pinning and should be tested against API updates before deploying new versions.
- **TensorFlow Lite Updates:** TFLite runtime updates may affect model loading behavior or inference performance. Model compatibility should be verified when upgrading the TFLite dependency.

6.3 Perfective Maintenance

Perfective maintenance improves the system beyond its current capabilities in response to user feedback or new research findings. Planned preventive maintenance activities include:

- **Model Retraining:** As more users optionally contribute anonymized behavioral data for research, the ML models can be retrained and improved. Lightweight model updates (parameter threshold updates) can be delivered as configuration updates without requiring a full application release.
- **Additional Sensing Modalities:** The architecture supports the future integration of new sensor sources such as ambient sound level monitoring, typing rhythm analysis, or calendar-based routine inference. Each new modality can be added as an independent Background Service without architectural changes to existing components.
- **Expanded Wearable Support:** Future versions may support additional wearable ecosystems beyond Samsung Galaxy Watch, such as Fitbit or Garmin, by extending the Health Connect Bridge component.
- **Localization:** The current implementation is primarily designed for English-language text and speech processing. Future perfective maintenance should add support for Turkish and other languages, particularly for the NLP and ASR components.
- **UI/UX Improvements:** User feedback from the demonstration sessions and future user studies should drive iterative improvements to the dashboard visualizations, onboarding clarity, and recommendation quality.

6.4 Database Migration Protocol

The Room database currently operates at Version 8. All future schema changes must be accompanied by a corresponding Migration object that specifies the SQL transformation required to upgrade from the previous version. Migrations are tested in isolation before inclusion in a release. The migration history is documented in the AppDatabase migration registry and reviewed during each maintenance cycle.

6.5 Dependency Management

Third-party dependencies (Flutter packages, Android Gradle dependencies, ONNX Runtime, TFLite) are managed through their respective package manifests (pubspec.yaml and build.gradle). Dependency versions should be reviewed every 6 months and updated incrementally. Security advisories for dependencies must be monitored, with critical security updates applied within 30 days of disclosure.

6.6 User's Manual and Installation Instructions

The following is a summary of the installation and operational instructions for the ScubaMind application. A complete User's Manual is provided as a separate accompanying document.

6.6.1 System Requirements

- Android 11 (API level 30) or higher
- At least 3 GB RAM recommended for ASR inference
- At least 500 MB of available internal storage
- Internet connectivity is required for the LLM Companion feature only
- Samsung Galaxy Watch (optional) for wearable physiological data

6.6.2 Installation

1. Transfer the ScubaMind APK file to the Android device or make it available through a direct download link.
2. On the device, navigate to Settings > Security and enable 'Install from Unknown Sources' (or grant permission when prompted during installation).
3. Open the APK file and tap 'Install'.
4. Once installed, open the ScubaMind application from the app drawer.

6.6.3 First Launch and Onboarding

5. Upon first launch, the Onboarding screen will guide you through the setup process.
6. Grant the requested permissions: Physical Activity (step counter), Location (mobility tracking), Microphone (voice recording), and Usage Stats (app usage monitoring).
7. Complete the baseline profile by providing your age and answering the initial PHQ-8 questionnaire.

8. If you wish to use the specialized keyboard for text analysis, follow the instructions to enable it in Android Settings > Accessibility > Installed Services.
9. Optionally connect your Samsung Galaxy Watch through the Health Connect integration by granting the relevant Health Connect permissions.

6.6.4 Daily Usage

After onboarding, ScubaMind runs background sensing services automatically based on your permission settings. Open the app at any time to view your behavioral dashboard on the Home screen. Use the Chat feature to interact with the AI companion. Record voice journals through the Voice Recordings section. Complete weekly PHQ-8 check-ins when prompted. Manage your data and permissions at any time through Profile > Privacy & Data.

6.6.5 Data Deletion

To permanently delete all data stored by ScubaMind, navigate to Profile > Privacy & Data > Delete All Data. This operation is irreversible and will cryptographically destroy all stored behavioral records by wiping the encryption keys from the Android Keystore.

7. Other Project Elements

7.1 Consideration of Various Factors in Engineering Design

The design of ScubaMind was shaped not only by technical requirements but also by broader engineering considerations related to public well-being, safety, ethics, privacy, and sustainability.

7.1.1 Constraints

Public Health

Public health is one of the most influential factors in the design of ScubaMind. Depression and related mental health disorders represent a major global health challenge. The system supports earlier awareness of behavioral changes that may be associated with mental health risks through continuous passive monitoring. However, the system is not designed as a diagnostic medical tool; outputs are intentionally framed as behavioral risk insights rather than medical conclusions, supporting public health responsibly while avoiding unsafe overclaiming.

Public Safety

Public safety constraints require that the system must not harm users through inaccurate interpretations, inappropriate recommendations, or privacy violations. All outputs use non-diagnostic language. The application guides users toward professional support rather than acting as a replacement for mental health professionals. The LLM Companion's Safety and Guardrail Layer monitors for high-risk intent indicators and triggers emergency guidance protocols when necessary.

Public Welfare

ScubaMind contributes to public welfare by making mental health-related behavioral insights accessible through consumer devices. The system provides a lower-barrier awareness tool that may help users understand their own behavioral patterns without requiring expensive or infrequent clinical visits.

Global Factors

Mental health is a universal issue, but language resources, digital infrastructure, wearable ecosystems, and device capabilities vary across regions. The current implementation is constrained by English-centered language processing and Android-based mobile environments. The modular architecture is designed to allow future adaptation for other languages and ecosystems.

Cultural Factors

Cultural differences influence how users interpret behavioral signals and mental health outputs. ScubaMind avoids culturally rigid assumptions in its analytical models and user-facing design. Wording, recommendations, and visual communication are designed to remain neutral, respectful, and non-stigmatizing.

Social Factors

Privacy concerns, trust, and stigma around mental health significantly influenced the system design. The adoption of a local-first architecture directly addresses social concerns by reducing the feeling of surveillance and strengthening user trust. The UI/UX is designed to be calming and non-intrusive for users experiencing stress, anxiety, or low motivation.

Environmental Factors

Continuous background sensing and on-device ML processing create risks of battery drain and device heating. This influenced the design toward energy-efficient background services, optimized data collection intervals, and lightweight quantized AI models. Unnecessary inference cycles are minimized, and only the required data is stored locally.

Economic Factors

As a Senior Design Project, the system is developed with limited resources, encouraging the use of open-source tools (TFLite, ONNX Runtime, Room, Flutter) and lightweight frameworks. Optional cloud components (Gemini API) are designed to be easily substituted or disabled to manage operational costs in future deployments.

Factor	Effect Level	Explanation
Public Health	Very High	Directly addresses mental health awareness through behavioral pattern monitoring and early risk insights.
Public Safety	High	Requires non-diagnostic outputs, responsible recommendations, and emergency guidance protocols.
Public Welfare	High	Improves accessibility to behavioral self-awareness tools through consumer smartphones and wearables.

Global Factors	Medium	Mental health is global, but language and device limitations constrain current universal applicability.
Cultural Factors	Low-Medium	Cultural differences affect insight interpretation; system design avoids culturally specific assumptions.
Social Factors	High	Privacy concerns and mental health stigma directly motivated the local-first architecture.
Environmental Factors	Medium	Continuous sensing and on-device AI require energy-efficient design and optimized model usage.
Economic Factors	Medium	Limited development resources and future API costs drive open-source and modular design choices.

Table 5 – Impact Levels of Various Engineering Design Factors on ScubaMind

7.1.2 Standards

IEEE Standards

The system design process follows principles inspired by IEEE 12207 (Software Life Cycle Processes) and IEEE 1016 (Software Design Description). Documentation structure, subsystem decomposition, and interface definition follow these principles to ensure clarity and traceability between requirements and design decisions.

ISO Standards

The design takes inspiration from the ISO/IEC 25010 Software Quality Model, which defines quality characteristics including reliability, usability, security, maintainability, and performance efficiency. These attributes directly influenced ScubaMind's design goals and test case criteria.

UML Standards

Unified Modeling Language (UML) standards are used for visualizing the system architecture and subsystem interactions. The design documentation includes system architecture diagrams, class diagrams, sequence diagrams, activity diagrams, use case diagrams, and data flow diagrams following OMG UML conventions.

Healthcare Data Standards (FHIR / HL7)

The system considers FHIR (Fast Healthcare Interoperability Resources) as a reference standard for structured health data representation. While ScubaMind does not currently implement a full FHIR integration, the data model is designed to allow future alignment with FHIR resource structures for potential interoperability with clinical systems.

KVKK (Turkish Personal Data Protection Law)

Given the sensitive nature of mental health data and the system's potential deployment in Turkey, the design considers compliance with KVKK regulations. The local-first architecture, explicit user consent mechanisms, and Data Purge feature support KVKK requirements for data minimization, user consent, and the right to erasure.

Object-Oriented Design Principles

The ScubaMind system follows object-oriented design principles, including encapsulation, modularity, and separation of concerns. Each subsystem operates as an independent module with clearly defined interfaces, simplifying maintenance and future extension.

Android Accessibility Guidelines

The application follows Android's material design and accessibility guidelines, including minimum touch target sizes (48 x 48 dp), adequate color contrast, and screen reader compatibility. These standards are validated through the TC-UX03 test case using the Android Accessibility Scanner.

7.2 Ethics and Professional Responsibilities

The ScubaMind project operates at the intersection of software engineering, artificial intelligence, and mental health, a domain that carries significant ethical responsibilities. The team recognized and fulfilled several categories of ethical obligations throughout the project lifecycle.

7.2.1 Responsibility for User Well-Being

The most fundamental ethical responsibility in this project is ensuring that the system does not cause harm to users. Mental health applications carry a unique risk: users in vulnerable states may place excessive trust in AI-generated outputs. To address this, the team made a principled design decision to frame all system outputs as behavioral awareness indicators rather than clinical diagnoses. The application consistently encourages users to seek professional support and does not attempt to replace clinical judgment. The LLM Companion includes safety guardrails specifically to detect distress signals and redirect users to appropriate emergency resources.

The team also considered the psychological impact of presenting risk scores. A risk score that is poorly communicated could cause unnecessary anxiety or, conversely, false reassurance. This drove the decision to display scores alongside explanatory context and to use qualitative labels (e.g., 'Your activity levels are lower than your usual pattern') rather than raw numerical values alone.

7.2.2 Privacy and Data Ethics

Collecting behavioral, physiological, and potentially linguistic data from users creates profound privacy responsibilities. The team proactively adopted a Privacy-by-Design approach from the earliest stages of system architecture, not merely as a compliance exercise but as an ethical commitment. The local-first architecture ensures that sensitive raw data never leaves the user's device. The specialized keyboard (IME) collects text across applications, which represents particularly sensitive data; this feature requires explicit user enablement and can be disabled at any time through clearly visible settings.

The data purge mechanism (Right to be Forgotten) was treated as a first-class feature rather than an afterthought, ensuring users can permanently and cryptographically destroy their data at any time. The team also ensured that the optional research participation feature, which uploads anonymized data to a research backend, requires explicit opt-in consent and clearly communicates what data is shared.

7.2.3 Transparency and Informed Consent

Users have a right to understand what data the system collects and how it is used. The onboarding flow was designed to clearly explain each requested permission and its purpose before requesting it. Privacy settings are accessible at any time through the Profile section. The team ensured that the application does not collect any sensor data before the user has completed the consent and onboarding process, and that all collection can be selectively disabled.

7.2.4 Avoiding Harm Through AI Model Outputs

AI models trained on population-level data may not generalize equally to all individuals. A model that performs well on average may produce misleading outputs for specific demographic groups, cultural backgrounds, or behavioral patterns not well-represented in training data. The team recognized this limitation and explicitly designed the system outputs as supportive indicators rather than definitive assessments. The PHQ-8 questionnaire is used as a calibration reference rather than a direct diagnostic output, maintaining the appropriate boundary between a decision-support system and a clinical tool.

7.2.5 Professional Responsibility in Documentation and Testing

A professional responsibility exists to deliver software that has been adequately tested and documented. The team fulfilled this responsibility by developing comprehensive functional and non-functional test cases covering all major system components, including edge cases (silent audio inference, ASR crash recovery, data persistence under unexpected termination) and security scenarios (unauthorized database access, plain-text storage prevention). Documentation was maintained throughout the project, including this Final Report, which provides a complete and accurate account of the system as built.

7.2.6 Responsible Use of External AI Services

The integration of the Gemini API introduces a dependency on a third-party AI service. The team took responsibility for ensuring that only anonymized, high-level behavioral context is transmitted to this external service, never raw behavioral logs, voice recordings, or identifiable user data. The prompt construction process was designed to minimize the information shared while still enabling meaningful, supportive responses.

7.3 Teamwork Details

The development of the ScubaMind system was carried out through a coordinated and collaborative teamwork process. Responsibilities were distributed according to the technical expertise, experience, and interests of each team member. Regular communication and structured meetings allowed the team to synchronize development across multiple subsystems and maintain alignment with the overall system architecture.

7.3.1 Contributing and Functioning Effectively on the Team

Each team member made distinct technical contributions to the ScubaMind architecture while maintaining coordination with the rest of the team.

Merve Güleç contributed to the behavioral data analysis and modeling components. She worked on GPS-based behavioral data processing and developed machine learning models using the StudentLife dataset to analyze behavioral patterns related to mental well-being. Her work supported the activity recognition and behavioral modeling components, contributing to the analytical layer that interprets passive sensing data. She also took the lead in preparing documentation sections related to behavioral analysis.

Metin Çalışkan contributed to the speech analysis components. His work focused on prosody-based analysis and extracting speech-related acoustic features (pitch variance, speech rate, intonation) that serve as vocal biomarkers for emotional state assessment. He also contributed to NLP-related components of the system architecture and assisted in integrating speech features into the multimodal analysis pipeline.

Muhammed Fatih Başal focused on the infrastructure required for integrating machine learning components into the overall system. He established model pipelines, prepared the technical environment for TFLite and ONNX model deployment, and ensured that analytical models operated as part of a consistent on-device processing pipeline. He also contributed to the design of the activity modeling components and the overall detailed system architecture.

Murathan Işık worked primarily on the mobile interaction layer. He adapted an externally sourced keyboard framework, modifying it to meet ScubaMind's requirements for the specialized IME keyboard component. He also contributed to user interface improvements to enhance usability and interaction quality throughout the mobile application.

Yiğit Koşum contributed to the conversational AI functionality. He implemented the Chat screen, integrated the LLM-based conversational interaction component using the Gemini API, and developed the contextual prompt construction system. He also contributed to project coordination, system planning, and the mobile application infrastructure in earlier project phases.

7.3.2 Helping Create a Collaborative and Inclusive Environment

Throughout the project, the team maintained a collaborative and inclusive working environment in which all members actively participated in discussions and decision-making processes. Weekly group meetings were held regularly to discuss project progress, evaluate technical challenges, and coordinate upcoming development tasks. Important architectural and implementation decisions were made collectively after consulting all team members.

Project documentation and reports were prepared collaboratively, with all members contributing to reviewing and refining content. Rather than assigning documentation tasks exclusively to specific individuals, the team operated with shared ownership over documentation quality, ensuring consistency across sections written by different members.

The team maintained open communication through a shared messaging channel and regular in-person meetings. When individual members encountered technical obstacles, the team addressed them collectively, drawing on the different expertise of

each member. This collaborative problem-solving approach prevented individual bottlenecks from blocking overall project progress.

Presentation preparation was also a collective effort. All members contributed to organizing materials, improving architectural explanations, and preparing visual content. The project demonstration was attended by all team members, who supported each other in explaining different aspects of the system.

7.3.3 Taking Lead Role and Sharing Leadership on the Team

Leadership responsibilities were distributed across subsystems and development stages, allowing the team to manage the complexity of the system effectively while ensuring each major component was guided by a member with relevant expertise.

Merve Güleç took the leadership role in behavioral modeling and guided the development of GPS-based behavioral analysis and activity modeling components. Her domain expertise in machine learning applications shaped the analytical approach of the AI Module.

Metin Çalışkan led the development of the speech prosody analysis component and contributed to the integration of speech-based behavioral signals into the multimodal analysis framework.

Muhammed Fatih Başal led the development of the machine learning infrastructure, ensuring that model training, evaluation, and deployment pipelines were consistent and reproducible across the development environment.

Murathan Işık led the development of the keyboard interaction module and took ownership of UI/UX improvements across the mobile application.

Yiğit Koşum led the integration of the conversational AI module and also served a coordinating role in the early phases of the project, helping to align the team's technical planning with the academic milestones.

7.3.4 Meeting Objectives

At the outset of the project, the team established a set of objectives and milestones in the project plan, documented in the Analysis Report. The following table reflects the degree to which these objectives were achieved at the time of the Final Report submission.

Milestone / Objective	Status	Notes
Architecture design and requirements documentation (Analysis Report)	Fully Met	Completed and submitted on schedule.
Detailed system design documentation (Detailed Design Report)	Fully Met	Completed with subsystem decomposition, diagrams, and test cases.
Passive behavioral sensing (steps, sleep, location, phone usage)	Fully Met	All four background services are implemented and functional.

On-device ASR (Whisper-tiny via ONNX Runtime)	Fully Met	SpeechToTextModel implemented with silence detection and error handling.
On-device ML inference (TFLite behavioral analysis)	Substantially Met	Core pipeline functional model performance requires further calibration with real-world data.
Prosody analysis feature	Partially Met	Acoustic feature extraction implemented; full integration into risk score pipeline in progress.
LLM Companion (Gemini API chat integration)	Fully Met	Chat interface with contextual prompt building implemented and functional.
Specialized IME keyboard	Substantially Met	Keyboard adapted and functional; full linguistic analysis pipeline integration ongoing.
AES-256-encrypted local database	Fully Met	Room + SQLCipher implemented with Android Keystore key management.
Privacy and data sovereignty features	Fully Met	Permission controls, data purge, and privacy settings implemented.
Wearable integration (Samsung Health Connect)	Partially Met	Health Connect Bridge implemented; full wearable data pipeline depends on device availability for testing.
Comprehensive testing (functional + non-functional)	Substantially Met	All test cases defined and executed; some performance metrics pending final device testing.
Final Report documentation	Fully Met	This document.

Table 6 – Milestone and Objective Status

Overall, the core functional objectives of the project were met or substantially met. The areas that were only partially completed, full prosody analysis pipeline integration, and wearable testing represent planned work that extends beyond the current project scope and are identified as future work in Section 8.

7.4 New Knowledge Acquired and Applied

The ScubaMind project required the team to acquire and apply new knowledge across several technical and non-technical domains. The following describes the key areas of new knowledge and the learning strategies used to acquire them.

7.4.1 On-Device Machine Learning and ONNX Runtime

Prior to this project, team members had experience with traditional machine learning training pipelines but limited experience with mobile deployment constraints. The team acquired new knowledge about TensorFlow Lite and ONNX Runtime for Android, including how to convert and optimize models for mobile inference, how to

manage memory and NPU/GPU resources on Android devices, and how to run inference asynchronously to prevent UI blocking.

Learning strategies included studying official TFLite and ONNX Runtime documentation, reviewing existing open-source mobile ML projects, and conducting iterative experiments to benchmark inference performance on physical devices. The team specifically learned how to use the Whisper model in ONNX format for on-device automatic speech recognition, a capability that was new to all team members.

7.4.2 Digital Phenotyping and Behavioral Signal Processing

The concept of digital phenotyping, using smartphone sensor data to infer behavioral and mental health indicators, was largely new to the team before this project. The team studied relevant academic literature, including the StudentLife dataset papers and digital phenotyping research by Onnela and Rauch, to understand which behavioral signals are most predictive of mental health changes and how they should be extracted and processed.

Learning strategies included a systematic literature review, analysis of existing datasets (StudentLife), and consultation with the project advisor. The team learned to compute behavioral biomarkers such as Radius of Gyration, Routine Index, and phone usage patterns, and to integrate these into a unified multimodal representation.

7.4.3 Flutter Development and Flutter-Native Communication

While some team members had prior Android development experience, the Flutter framework and Dart language were new to most of the team. The team learned Flutter's widget system, BLoC state management pattern, and navigation architecture. Importantly, the team acquired new knowledge about Flutter Method Channels, the mechanism for communicating between Dart and native Kotlin/Java code, which is essential for an application that bridges high-level UI with low-level sensor access.

Learning strategies included completing Flutter official documentation tutorials, studying existing Flutter open-source applications, and iterative prototyping with code review sessions among team members.

7.4.4 Android Background Services and System-Level APIs

Implementing continuous background sensing on Android requires deep knowledge of Android's service lifecycle management, foreground service requirements, battery optimization restrictions, and permission model. The team acquired significant new knowledge about Android's Foreground Service API, BootReceiver broadcast handling, UsageStatsManager, Android Keystore System, and Health Connect API, several of which were entirely new to the team.

Learning strategies included studying Android developer documentation, experimenting with API behavior across different Android versions, and reviewing open-source projects that implement similar background sensing functionality.

7.4.5 Privacy-Preserving System Design

Designing a system that handles sensitive mental health data required the team to acquire new knowledge about privacy engineering principles, Android security mechanisms (SQLCipher, EncryptedSharedPreferences, Keystore), and the Privacy-by-Design framework. The team also studied relevant data protection regulations, including GDPR principles and KVKK, to understand their implications for system design.

Learning strategies included reviewing OWASP Mobile Security guidelines, studying the Android Security Best Practices documentation, and examining existing privacy-focused health applications as reference examples.

7.4.6 Large Language Model Integration and Prompt Engineering

Integrating the Gemini API required the team to learn about LLM API interaction patterns, prompt engineering for mental health supportive dialogue, and the design of safety guardrails for AI-powered conversational systems. The team learned how to construct contextual system prompts that guide the model toward empathetic and supportive responses while preventing harmful outputs.

Learning strategies included experimenting with prompt variations, reviewing Gemini API documentation, and studying examples of responsible AI deployment in healthcare-adjacent applications.

8. Conclusion and Future Work

8.1 Conclusion

This Final Report presents the complete design, implementation, and evaluation of ScubaMin, a privacy-preserving mobile mental health monitoring system developed as part of the Bilkent University Senior Design Project course.

ScubaMind demonstrates that it is feasible to build a continuous behavioral monitoring system on consumer smartphones that respects user privacy through a local-first architecture, where all sensitive data is collected, stored, and analyzed on the user's device. The system successfully integrates passive sensing (step counting, sleep monitoring, GPS-based mobility tracking, phone usage statistics), on-device machine learning (TFLite behavioral analysis, Whisper-based ASR), wearable physiological data (via Samsung Health Connect), and an LLM-powered conversational companion (Gemini API) within a single cohesive Flutter application.

The core functional objectives of the project were met: the sensing pipeline collects and aggregates multimodal behavioral data into the unified daily data model, the AI module computes a weekly risk score and generates behavioral insights, the voice recording and transcription features operate end-to-end on-device, the LLM companion engages users in reflective dialogue with contextual awareness of their behavioral state, and the privacy features (encrypted database, permission controls, data purge) protect user data in accordance with Privacy-by-Design principles.

The project also produced significant learning outcomes for all team members across mobile development, on-device AI deployment, behavioral signal processing, privacy engineering, and responsible AI design in the mental health domain.

8.2 Future Work

Several directions for future development were identified during the project:

- **Full Prosody Analysis Integration:** The prosody feature extraction pipeline should be fully integrated into the multimodal risk score computation, enabling vocal biomarkers to contribute to the weekly risk assessment alongside mobility and usage data.
- **Model Training with Real-World Data:** As users optionally contribute anonymized behavioral data for research, the behavioral analysis models should be retrained on this enriched dataset to improve prediction accuracy and personalization.
- **Expanded Wearable Ecosystem Support:** Future versions should extend wearable integration beyond the Samsung Galaxy Watch to support other popular wearable platforms through the Health Connect API.
- **Multilingual Support:** Adding Turkish and other language support to the NLP and ASR components would significantly broaden the system's accessibility.
- **Longitudinal User Study:** A formal longitudinal user study with clinical collaboration would enable proper validation of the system's behavioral indicators against clinical assessments.
- **iOS Platform Support:** Extending the application to iOS, leveraging Apple HealthKit for physiological data, would increase the potential user base.
- **Report Generation:** Implementing the planned PDF report generation feature would allow users to share behavioral trend summaries with healthcare providers, supporting the system's role as a decision-support tool.
- **Enhanced Explainability:** Adding more detailed explanations of what drives the risk score (e.g., 'Your risk score is influenced primarily by reduced mobility this week') would improve user understanding and trust in the system.

9. Glossary

AES-256: Advanced Encryption Standard with a 256-bit key. The industry-standard symmetric encryption algorithm used by SQLCipher to secure the local database in ScubaMind.

Android Keystore System: A secure, hardware-backed storage environment (TEE/StrongBox) used to protect the dynamically generated cryptographic keys required for database encryption.

ASR (Automatic Speech Recognition): The technology and models used for transcribing spoken audio from journaling sessions into raw text for further analysis.

Background Service Orchestrator: A native Android component that manages the lifecycle of Foreground Services to ensure continuous, energy-efficient data acquisition without OS-level termination.

Behavioral Baseline: A user-specific reference profile constructed from historical behavioral data, used to interpret future deviations in the user's daily patterns.

Behavioral Insight: A system-generated interpretation of collected behavioral and physiological data, presented to the user to improve self-awareness rather than provide a clinical diagnosis.

BLoC (Business Logic Component): A state management pattern used in the Flutter layer to separate UI logic from business logic through reactive event-driven streams.

BootReceiver: An Android broadcast receiver component responsible for restarting necessary background services (based on user privacy settings) when the device boots up.

DAO (Data Access Object): An interface utilized by the Room database library to abstract and manage SQL operations.

Data Purge (Right to be Forgotten): A privacy mechanism that securely wipes the AppDatabase and resets EncryptedSharedPreferences, rendering all stored data cryptographically unrecoverable.

Digital Phenotyping: The passive and continuous collection of behavioral and physiological signals from personal digital devices to analyze patterns related to mental well-being.

Embedded AI Core: The on-device machine learning component of ScubaMind, relying on TensorFlow Lite and ONNX Runtime to analyze multimodal signals without transmitting sensitive data to the cloud.

EncryptedSharedPreferences: A secure Android storage mechanism used to safely store the dynamically generated UUID passphrase required to unlock the SQLCipher database.

FHIR (Fast Healthcare Interoperability Resources): A healthcare data standard for structured and interoperable electronic health information exchange.

Foreground Service: A specialized Android service class tied to a persistent notification, ensuring critical data collection tasks are not killed by the operating system.

Health Connect Bridge: A software integration layer synchronizing physiological metrics (HRV, Sleep Stages) from Samsung Health and wearable devices into the local database.

HRV (Heart Rate Variability): A physiological indicator representing the variation in time between heartbeats, used as a biomarker for stress and autonomic nervous system balance.

IME (Input Method Editor): A specialized Android keyboard component custom-built for ScubaMind to capture raw text input across apps for linguistic and sentiment analysis.

KVKK: The Turkish Personal Data Protection Law governing the processing, storage, and protection of personal data.

Late-Fusion Neural Network: A machine learning architecture where separate data streams are processed independently and synthesized at the final stage into a unified Risk Score.

LLM Companion: A cloud-based module using the Gemini API to provide supportive diary-style conversational guidance based on anonymized behavioral context.

Local-First Architecture: A core privacy-preserving design principle where all sensitive user data is collected, encrypted, and processed directly on the smartphone, minimizing cloud dependencies.

Method Channel: A communication bridge used to pass messages asynchronously between the Flutter UI codebase (Dart) and the Native Android backend (Kotlin).

Multimodal Fusion: The process of combining multiple distinct sources of behavioral and physiological data into a single comprehensive analytical model.

NPU (Neural Processing Unit): Specialized hardware embedded in modern smartphones designed to accelerate machine learning inference.

On-Device Inference: The execution of trained machine learning models locally on the mobile device, eliminating the need to send raw behavioral data to external servers.

PHQ-8 (Patient Health Questionnaire-9): A standard 9-item clinical depression questionnaire used in ScubaMind as a baseline reference point and research label.

Privacy by Design: A system design principle that integrates privacy protection into the architecture from the beginning.

Prosody Analysis: The extraction and evaluation of acoustic features from voice recordings (pitch variance, speech rate, intonation) serve as vocal biomarkers for emotional states.

Radius of Gyration: A computed mobility metric indicating the spatial variance of a user's movement from their central or home location.

Risk Score: A continuous or categorized behavioral indicator generated by the AI Module to reflect potential changes in a user's mental well-being.

Room Persistence Library: An official Android Jetpack library providing an abstraction layer over SQLite for structured and encrypted local database management.

Routine Index: A derived behavioral biomarker reflecting the regularity and predictability of a user's daily mobility traces and activities.

SQLCipher: An open-source extension to the SQLite database library providing transparent 256-bit AES encryption.

TFLite (TensorFlow Lite): A framework for deploying lightweight machine learning models on mobile and edge devices with low latency.

UnifiedDataEntity: The core analytical database table aggregating daily metrics (steps, sleep, PHQ scores, mobility, text fragments) into a single row per day.

UsageStatsManager API: A native Android interface for tracking digital behavior patterns, including app usage duration and screen-on/off events.

VoiceRecordingEntity: A database schema element storing metadata for local audio journals, mapping file paths and durations to transcription data.

10. References

- [1] World Health Organization. Depression. World Health Organization. Available: <https://www.who.int/health-topics/depression>. Accessed: 2026.

- [2] R. C. Kessler and E. J. Bromet, "The epidemiology of depression across cultures," *Annual Review of Public Health*, vol. 34, pp. 119–138, 2013.
- [3] J. P. Onnela and S. L. Rauch, "Harnessing smartphone-based digital phenotyping to enhance behavioral and mental health," *Neuropsychopharmacology*, vol. 41, pp. 1691–1696, 2016.
- [4] R. Wang, F. Chen, Z. Chen, T. Li, G. Harari, S. Tignor, X. Zhou, D. Ben-Zeev, and A. T. Campbell, "StudentLife: Assessing mental health, academic performance, and behavioral trends of college students using smartphones," in *Proc. ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp)*, 2014. doi: 10.1145/2632048.2632054.
- [5] Moodpath GmbH. Moodpath / MindDoc Mental Health Application. Available: <https://www.minddoc.com/>. Accessed: 2026.
- [6] Mindstrong Health. Mindstrong Digital Mental Health Platform. Available: <https://www.linkedin.com/company/mindstrong/>. Accessed: 2026.
- [7] Apple Inc. Apple HealthKit Documentation. Available: <https://developer.apple.com/healthkit/>. Google LLC. Google Fit Documentation. Available: <https://developers.google.com/fit>. Accessed: 2026.
- [8] IEEE, IEEE/ISO/IEC 12207-2017: Systems and Software Engineering Software Life Cycle Processes. IEEE, 2017.
- [9] IEEE, IEEE Std 1016-2009: IEEE Standard for Information Technology Systems Design Software Design Descriptions. IEEE, 2009.
- [10] ISO/IEC, ISO/IEC 25010:2011: Systems and Software Engineering Systems and Software Quality Requirements and Evaluation (SQuaRE) System and Software Quality Models. ISO, 2011.
- [11] OWASP Foundation. Mobile Application Security Testing Guide (MASTG). Available: <https://mas.owasp.org/MASTG/>. Accessed: 2026.
- [12] Android Developers. Improve Your App's Security. Available: <https://developer.android.com/privacy-and-security/security-best-practices>. Accessed: 2026.
- [13] Android Developers. Health Connect Documentation. Available: <https://developer.android.com/health-and-fitness/health-connect>. Accessed: 2026.
- [14] Google LLC. Gemini API Documentation. Available: <https://ai.google.dev/gemini-api/docs>. Accessed: 2026.
- [15] Microsoft. ONNX Runtime Mobile Documentation. Available: <https://onnxruntime.ai/docs/get-started/with-mobile.html>. Accessed: 2026.
- [16] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research*, vol. 202, pp. 28492–28518, 2023.

11. Appendix — User Manual and Project Website

The ScubaMind user manual and project website were prepared as supporting documentation for the final prototype. The user manual provides step-by-step guidance for using the application, including onboarding, permission management, the Home Dashboard, risk breakdown, goals, notifications, Scuby AI Chat, privacy controls, report generation, troubleshooting, and safety disclaimers. The project website presents the project overview, motivation, main features, screenshots, team information, and supporting materials.

Resource	Description	Link
User Manual	Step-by-step guide for using ScubaMind and understanding its main features.	https://docs.google.com/document/d/1v0SHsUsD-3S6vxXGfZAtL1hniwILQdtC_38yF54J_VU/edit?usp=sharing
Project Website	Online page presenting the project overview, main features, screenshots, team information, and supporting materials.	https://scuba-mental-health.netlify.app/